

ASSESSING THE CAPABILITY OF LARGE LANGUAGE MODELS IN GENERATING
ASP-COMPATIBLE LOGIC FORMS FROM HUMAN NARRATIVES

Amritmani Aryal

A THESIS

Presented to the Faculty of Miami University in partial
fulfillment of the requirements
for the degree of

Master of Science

Department of Computer Science & Software Engineering

The Graduate School
Miami University
Oxford, Ohio

2026

Dr. Daniela Inclezan, Advisor
Dr. Hakam Alomari, Reader
Dr. Alan Ferrenberg, Reader

©
Amritmani Aryal
2026

Abstract

Meaningful narrative understanding by AI systems requires both deep language comprehension and structured reasoning. This thesis investigates how effectively Large Language Models (LLMs) can generate structured, logic-compatible representations, called Logic Forms, from unstructured natural language narratives, for use as inputs to Answer Set Programming (ASP) systems.

We present a systematic comparative evaluation of three state-of-the-art LLMs—GPT-5, Gemini 2.5 Pro, and DeepSeek V3.1—across a matrix of prompting conditions that varied the number of worked examples (0, 5, and 10 shots) and the presence of supplementary encoding heuristics. Using a curated dataset of 100 restaurant-domain narratives and a dual evaluation protocol combining automated syntactic validation with manual semantic annotation, we benchmark both the parsability and the faithfulness of model-generated ASP encodings.

Our results reveal that model performance is both prompting-dependent and model-specific. Under the richest configuration, GPT-5 and Gemini 2.5 Pro achieved 83% and 79% semantic accuracy respectively, with 100% syntactic validity across all three models. However, the path to reliability differed markedly by architecture: Gemini showed a 37-point improvement when encoding heuristics were added to a moderate exemplar set, while DeepSeek’s performance exhibited a U-shaped curve, degrading under intermediate prompting before recovering in the richest condition. Notably, models occasionally produced encodings judged *better than the human-annotated ground truth*, suggesting a collaborative rather than purely assistive role for LLMs in knowledge engineering workflows.

We also present a taxonomy of six distinct failure modes ranging from missing syntactic delimiters to unsafe variables, character-level hallucinations, and predicate invention, that characterize the boundary between successful extraction and model error. These findings offer practical, model-aware guidelines for leveraging LLMs as reliable structured information extractors for formal reasoning systems and identify critical directions for future work at the intersection of neural and symbolic AI.

Table of Contents

Abstract	ii
Abstract	ii
List of Figures	vii
1 Introduction	1
1.1 Motivation	1
1.2 Contributions	2
2 Background & Related Work	4
2.1 Narrative Understanding	4
2.1.1 Importance of Narrative Understanding in Computer Systems	4
2.1.2 Challenges in Computational Narrative Understanding	4
2.1.3 Existing Approaches to Narrative Representation	5
2.1.4 Why Narratives Matter for Knowledge Representation and Reasoning	7
2.2 Knowledge Representation	7
2.2.1 Role of KR in AI	7
2.2.2 Techniques for Knowledge Representation	8
2.3 Logic Programming & Answer Set Programming (ASP)	10
2.3.1 Overview of Logic Programming & Its Role in Reasoning Tasks	10
2.3.2 Key Features & Advantages of Answer Set Programming (ASP)	11
2.3.3 Examples of Using ASP in Narrative Understanding and Decision-Making Systems	12
2.4 Large Language Models (LLMs) in Natural Language Processing	13
2.4.1 Evolution of LLMs and Their Capabilities	13
2.4.2 Specific Applications of LLMs in Extracting Structured Information	14
2.4.3 Challenges LLMs Face in Narrative Understanding	14
2.4.4 Overcoming the Challenges	15
2.5 Bridging LLMs with Logic Programming	15
2.5.1 Conceptual Challenges and Technical Gaps	16
2.5.2 Existing Attempts to Integrate LLMs with Formal Reasoning	16
2.5.3 The Potential for Hybrid Models	17
2.6 Related Work: Comparative Studies and Gaps	17

2.6.1	Key Studies and Contributions:	17
2.6.2	Limitations of Current Methodologies and Research Scope	20
2.6.3	Research Gaps Addressed by This Work	21
3	Methodology and Experimental Setup	22
3.1	Research Questions	22
3.2	Data Preparation and Challenges	24
3.2.1	Data Sourcing and Extraction	26
3.2.2	Ground Truth Encoding	27
3.2.3	Challenges in Dataset Construction	27
3.3	Experimental Conditions and Prompt Design	29
3.3.1	Model Selection	29
3.3.2	Common Prompt Architecture	31
3.3.3	Condition: ZeroShot-V1	34
3.3.4	Condition: FewShot-5-Random-V1	35
3.3.5	Condition: FewShot-5-Random-V2	36
3.3.6	Condition: FewShot-10-Random-V1	37
3.3.7	Condition: FewShot-10-Random-V2	39
3.3.8	Condition: Manual-Prompting (Web Interface, Latest Model Versions)	39
3.3.9	Prompting Challenges	41
4	Results and Evaluation	43
4.1	Evaluation Metrics and Annotation Protocol	43
4.1.1	Syntactic Validity: Automated Clingo Parsing	43
4.1.2	Semantic Accuracy: Manual Annotation Protocol	44
4.2	Syntactic Evaluation Results	47
4.2.1	ZeroShot-V1	47
4.2.2	FewShot-5-Random-V1	48
4.2.3	FewShot-5-Random-V2	49
4.2.4	FewShot-10-Random-V1	51
4.2.5	FewShot-10-Random-V2	52
4.3	Discussion of Syntactic Results	52
4.3.1	Comparative Model Performance	53
4.3.2	Effect of Shot Count on Syntactic Correctness	53
4.3.3	Effect of the Supplementary Encoding Guide	54
4.3.4	Common Parser Errors and Failure Mode Taxonomy	55
4.4	Semantic Evaluation Results	56
4.4.1	ZeroShot-V1	57
4.4.2	FewShot-5-Random-V1	58
4.4.3	FewShot-5-Random-V2	60
4.4.4	FewShot-10-Random-V1	62
4.4.5	FewShot-10-Random-V2	63
4.4.6	Manual-Prompting (Web Interface)	65

4.4.7	Summary of Semantic Results	66
5	Conclusion and Future Work	69
5.1	Summary of Findings	69
5.2	Addressing the Research Questions	70
5.3	Broader Implications	71
5.4	Limitations	72
5.5	Future Work	73
5.6	Closing Remarks	73
A	Encoding Schema Specification	75
B	Supplementary Encoding Guide	79
B.1	Core Principle	79
B.2	Order Specifications	79
B.3	Inference Rules	79
B.4	Entity Representation	80
B.5	General Guidelines	80
	References	81

List of Figures

2.1	Simplified script-based representation for the Nicole narrative	5
3.1	Overview of the experimental pipeline, illustrating the flow from narrative input through prompting conditions and model inference to the two-stage evaluation process.	23
3.2	Overview of the data preparation pipeline. Three source corpora—Inclezan et al. (XML), ROCStories (CSV), and LLM-generated (plain text)—were processed and normalized into a unified JSON dataset of 100 restaurant-domain narratives, which then underwent manual ground-truth encoding to produce the final set of ASP logic forms.	25
3.3	A representative sample of the unified JSON format for two stories, shown in the IDE.	26
3.4	Overview of the experimental infrastructure, showing the flow from the input dataset through prompt construction and the OpenRouter API to the three primary models under consistent inference settings.	31

Chapter 1

Introduction

Narratives, especially those found in everyday stories, often contain complex structures, implied intentions, and subtle causal relationships that are intuitive for humans to understand but challenging for machines. For AI to engage in advanced applications—such as virtual assistants that respond based on context, intelligent storytelling, and decision-making in dynamic environments—it is essential for machines to interpret stories in a manner similar to human understanding. This deep interpretation often involves inferring implicit information and resolving ambiguities, tasks that benefit significantly from symbolic reasoning approaches. Systems based on logic programming, for example, excel at modeling the underlying rules, constraints, and causal dependencies that govern a narrative’s progression.

Currently, Large Language Models (LLMs) represent a major step forward in language comprehension, with models like GPT-4, Claude models from Anthropic, the Gemini family of models from Google, etc., capable of extracting insights from unstructured text [1, 2]. However, while LLMs demonstrate strong language comprehension, a key question is how effectively their outputs can serve as direct, structured input for deeper reasoning tasks performed by systems like logic programs.

1.1 Motivation

The motivation for this research stems from the complementary strengths and potential synergy between Large Language Models (LLMs) and logic programming systems like Answer Set Programming (ASP) for narrative understanding. While LLMs demonstrate powerful capabilities in processing unstructured text, they often lack the verifiable consistency and deep reasoning abilities inherent in formal logic systems. Conversely, ASP offers rigorous reasoning but typically requires structured knowledge bases that are challenging to create manually, especially from complex narratives. This research is motivated by the opportunity to bridge this gap by leveraging LLMs specifically for the task of automatically generating structured, ASP-compatible logic forms from narrative text, potentially overcoming the knowledge acquisition bottleneck for logic programming systems.

However, the reliability and quality of LLM outputs for this specific purpose remain largely unverified. Therefore, a key motivation is the critical need for systematic comparison: this research will evaluate how different state-of-the-art LLMs perform when tasked with extracting narrative elements and representing them in a precise ASP format, addressing a significant gap in the current literature. Furthermore, given the potential for errors or

syntactic inconsistencies in LLM outputs, this work is driven by the challenge of optimizing LLM performance for this task. We will investigate the effectiveness of prompt engineering and fine-tuning techniques specifically aimed at enhancing the semantic accuracy and syntactic validity of the generated logic forms, making them more suitable as direct inputs for ASP systems—an area currently underexplored for narrative contexts.

By systematically evaluating LLM capabilities and optimization techniques for this crucial extraction and translation step, this research aims to provide the foundational knowledge and practical methods needed to reliably generate high-quality inputs for formal reasoning systems, paving the way for more effective integration of LLMs and logic programming for deeper narrative analysis.

1.2 Contributions

This research focuses on the critical step of transforming unstructured narratives into structured representations suitable for formal reasoning systems, called *logic forms*. Specifically, this work contributes to the field in the following ways:

1. **Extraction Framework for ASP-Compatible Structured Data:** We propose a structured pipeline leveraging LLMs to transform unstructured narratives into logic program-compatible representations (e.g., predicates, temporal/causal relationships). This framework prioritizes outputs that serve as direct inputs for ASP systems, enabling downstream reasoning tasks without requiring manual restructuring.
2. **Comparative Analysis of LLMs for Narrative Extraction:** We conduct a systematic evaluation of state-of-the-art LLMs (e.g., GPT-4, Claude, Gemini) to benchmark their effectiveness in extracting narrative elements (events, relationships, intentions) from unstructured text. This analysis identifies model-specific strengths, limitations, and biases in capturing implicit or ambiguous story structures.
3. **Optimizing LLM Outputs via Prompt Engineering and Fine-tuning:** We investigate the impact of various prompt engineering strategies and explore the potential of fine-tuning techniques to improve the quality, reliability, and structural consistency of the narrative information extracted by LLMs, specifically aiming for outputs that align better with the requirements of logical formalisms like ASP.
4. **Formal Suitability Analysis of LLM Outputs for ASP Systems:** We analyze the nature and limitations of the structured outputs generated by current LLMs from narratives. This involves identifying common success patterns, failure modes (e.g., hallucinations, inconsistencies, structural errors), and assessing their overall readiness to serve as direct input for ASP systems without significant manual post-processing.

By addressing these objectives, this research aims to significantly advance the understanding of how LLMs can bridge the gap between unstructured natural language narratives

and structured knowledge representations required for formal reasoning. The primary focus is on evaluating and enhancing the LLM’s role as a structured information extractor for potential use in logic programming contexts, thereby contributing practical insights to the fields of Knowledge Representation, Natural Language Processing, and AI systems integration.

We specifically focus on narratives composed of sequences of events, as these have been shown to pose challenges for some LLMs in terms of reasoning and question answering [3]. Although this thesis leverages ASP for its strengths in representing and reasoning about commonsense knowledge, the central emphasis concerns evaluating LLMs and their effectiveness in generating logical forms from narrative text.

Chapter 2

Background & Related Work

This section will give a broad overview of the important concepts relevant to this research on using Large Language Models with Logic Programming for narrative understanding. It will also cover previous works related to our research, their strengths and limitations.

2.1 Narrative Understanding

Narratives, or stories, play a central role in how humans communicate, interpret events, and infer causality. They consist of sequences of events, intentions, and relationships, often involving multiple actors, so they are distinct from mere descriptions or lists of events [4]. In artificial intelligence as well, understanding narratives is a critical challenge due to their unstructured nature and the need to infer implicit information.

2.1.1 Importance of Narrative Understanding in Computer Systems

Narratives are essential in domains such as virtual assistants, automated planning, and intelligent storytelling. These systems must go beyond surface-level understanding to reason about intentions and relationships within the narrative context. For instance, in a restaurant scenario, understanding a customer’s actions (e.g., ordering food) requires recognizing both explicit actions (e.g., the verbal request for a specific dish) alongside implicit expectations (such as the intention to eat, the assumption that the kitchen will prepare the order, and the eventual need to pay)[5].

2.1.2 Challenges in Computational Narrative Understanding

Narratives are often multiple sentences or paragraphs long, requiring systems to maintain and process context over long sequences, which poses a significant challenge for computational narrative understanding. Additionally, many narrative elements, such as intentions or causal relationships, are not explicitly stated but rather implied through context, tone, or cultural knowledge, making them particularly difficult to extract and interpret. Furthermore, narratives generally have hierarchical structures of events and sub-events, which add layers of complexity that make it challenging to represent them comprehensively in a way that captures their full depth and interconnectedness. These factors—context retention, im-

implicit information, and structural complexity—collectively make it difficult for computational systems to achieve robust narrative understanding.

2.1.3 Existing Approaches to Narrative Representation

Early computational approaches relied on script-based models, which used pre-defined templates to represent stereotypical activities (e.g., dining at a restaurant) [6, 7, 8]. For instance, Mueller [8] discusses representing narratives like the following using such models:

Nicole went to a vegetarian restaurant. She ordered lentil soup. The waitress set the soup in the middle of the table. Nicole enjoyed the soup. She left the restaurant.

A script-based system processing this story would activate a “RESTAURANT” script. This script contains predefined roles (like Customer, Waiter, Food) and expected event sequences. As the system processes the narrative, it fills the slots of this script template with specific information extracted from the text. Figure 2.1 illustrates a simplified representation of the resulting knowledge state, capturing the key entities and the overall script context derived from Nicole’s story:

SCRIPT:TYPE	RESTAURANT
SCRIPT:LAST EVENT	LEAVE
RESTAURANT	'the vegetarian restaurant'
WAITER	'waitress'
CUSTOMER	'Nicole'
FOOD	'Lentil soup': 'Nicole'

Figure 2.1: Simplified script-based representation for the Nicole narrative

This structured representation, based on the predefined RESTAURANT template, allows the system to organize the information and potentially infer unstated actions common to the script (like Nicole eating the soup before leaving). While effective for rigid scenarios, these models struggled to generalize to real-world narratives due to their inability to handle variability, exceptions, or implicit context. Logic-based frameworks like Answer Set Programming (ASP) addressed this limitation by encoding narratives as formal rules, enabling structured reasoning about events, relationships, and intentions [9, 5, 10].

Following the logic-based approach, particularly Answer Set Programming (ASP), the same narrative about Nicole would be translated into a more granular, event-based logic form. This involves defining the entities and representing the sequence of actions as timed

events. Based on the representation style shown in [10], this could look like the following set of ASP facts:

```
% Entity Declarations
customer(nicole).
restaurant(veg_r).
food(lentil_soup).
waitress(waitress).
cook(cook1).

% Event Occurrences (using st_hpd(action, truth_value, time_step))
st_hpd(enter(nicole, veg_r), true, 0).
st_hpd(order(nicole, lentil_soup, waitress), true, 1).
st_hpd(put(waitress, lentil_soup, table), true, 2).
st_hpd(eat(nicole, lentil_soup), true, 3).
st_hpd(leave(nicole), true, 4).
```

In English, these ASP facts state:

- **Entities:** ‘nicole’ is a customer, ‘veg_r’ is a restaurant, ‘lentil_soup’ is food, ‘waitress’ is a waitress, and ‘cook1’ is a cook.
- **Events:**
 - **At time 0:** It is true that the action ‘nicole enters veg_r’ occurred.
 - **At time 1:** It is true that the action ‘nicole orders lentil_soup from the waitress’ occurred.
 - **At time 2:** It is true that the action ‘the waitress puts lentil_soup on the table’ occurred.
 - **At time 3:** It is true that the action ‘nicole eats lentil_soup’ occurred.
 - **At time 4:** It is true that the action ‘nicole leaves’ occurred.

This ASP representation breaks the narrative into discrete, timed events and explicitly defines the participating entities. Unlike the script model, which relies on a monolithic template, this structure allows for more flexible reasoning about sequences, causality, and potential deviations using ASP rules.

Modern Large Language Models (LLMs) excel at natural language processing tasks such as extracting entities, actions, and surface-level relationships from unstructured text. However, they are still not as great at performing complex reasoning tasks—such as inferring causal chains, resolving narrative inconsistencies, or deducing unstated intentions—which require deterministic, rule-based logic. To illustrate this limitation, consider a simple narrative tested by Yuheng Wang [3] against ChatGPT 3.5 for their research when this model

had just come out to the market:

Mary went to the kitchen. Mary got the apple. Mary got the ball. Mary got the book. Mary went to the bedroom .Mary went to the garden. Mary dropped the book.

When ChatGPT (3.5) was asked, “*Where is the apple?*” it often struggled to get the correct answer for this simple problem. However, it is important to mention that ChatGPT’s latest models (April 2025) get it right for this particular example. Furthermore, despite ongoing research and advancements aimed at improving reasoning and explainability in newer models, the often opaque *black-box* nature of current LLMs still makes it difficult to rigorously verify the logical soundness or consistency of their outputs compared to formal methods. This highlights the important role that logic programs, such as ASP, continue to play in complex reasoning tasks.

However, the reliability of LLMs in generating ASP-compatible inputs remains underexplored, particularly in dynamic or ambiguous narrative contexts. This gap underscores the need for systematic evaluation of LLMs’ capacity to bridge unstructured text with formal logic frameworks like ASP.

2.1.4 Why Narratives Matter for Knowledge Representation and Reasoning

Narratives provide a natural testbed for developing and evaluating reasoning systems [11]. They require systems to integrate knowledge representation (for encoding events and relationships) with reasoning capabilities (for inferring intentions or outcomes). This makes narratives an ideal domain for exploring the intersection of natural language processing and logic programming.

2.2 Knowledge Representation

Knowledge Representation (KR) is a foundational area in Computer Science focused on how information about the world is encoded in a format that enables machines to reason, infer, and act on it [12]. It involves designing data structures and procedures for storing and manipulating information in computer programs [13]. The goal is to represent human knowledge in a computer language that enables reasoning across various applications, including information search, simulation, and web semantic ontology description [14]. In the context of narrative understanding, KR serves as the bridge between unstructured natural language data and structured reasoning frameworks, such as logic programming.

2.2.1 Role of KR in AI

Knowledge representation (KR) plays a crucial role in artificial intelligence (AI) by enabling intelligent behavior in AI agents [15]. KR provides a systematic way to represent entities,

events, relationships, and rules about the world. It involves data structures and interpretive procedures that lead to knowledgeable behavior when used in programs [13]. KR is central to AI research, as it emphasizes the importance of expert knowledge and associated handling facilities in intelligent systems [16]. For narratives, KR ensures that the story’s elements—such as events, actors, and their relationships—are encoded in a format that a logic programming system can process.

2.2.2 Techniques for Knowledge Representation

- **Semantic Networks:** Graph-based structures used to represent knowledge in computational systems, particularly for modeling natural language understanding and hierarchical reasoning [17]. These networks consist of nodes representing concepts or word-sense meanings, connected by arcs denoting semantic relations [18]. Semantic networks are useful for capturing relationships but can lack the precision required for complex reasoning.
- **Frame-Based Systems:** Organize knowledge into schemas or frames, which represent stereotypical situations, such as dining at a restaurant. They were foundational in early narrative understanding, however they face limitations in handling uncertainty and noise [19]. Frame-based spoken language understanding is widely used in transactional dialog systems but is often limited to specific domains with well-defined semantic spaces [20]. Another challenge for frame-based systems is scalability, as many are implemented in main memory, limiting their capacity for large-scale applications [21].
- **Rule-Based Systems:** Rule-based systems encode knowledge as “if-then” rules that can be executed to infer new information or make decisions [22]. Rule-based systems have significant applications in various domains, including medical diagnosis, where they analyze patient data using inference rules [23]. They are commonly used in expert systems and decision support systems as they provide a straightforward way to represent logical relationships. They are easy to understand and implement for well-defined problems, and they are suitable for domains with clear, deterministic relationships. However, they struggle with uncertainty and require extensive rule definitions for complex domains. These systems often rely on shallow, experiential knowledge, which can lead to incorrect conclusions when applied to modified contexts [24]. Despite these drawbacks, rule-based approaches are still widely used in industry for information extraction, particularly by large vendors like IBM, SAP, and Microsoft [25].
- **Ontologies:** Ontologies represent knowledge as a set of concepts and categories within a domain and the relationships between them. Ontologies are formal representations of shared knowledge in a structured manner, enabling semantic interoperability and knowledge management [26]. They play a crucial role in various domains, including e-business, information retrieval, natural language processing, knowledge management, and the Semantic Web [26]. However, Ontologies face several limitations in their development and application. [27] highlights the scarcity of practical, non-toy ontologies,

attributing this to technical constraints and social factors. Brewster et al. (2004) question the assumption that ontologies are universally suitable for knowledge representation, emphasizing the need to critically examine their capabilities.

- **Probabilistic Graphical Models:** Probabilistic graphical models (PGMs) are powerful tools for representing complex probability distributions and reasoning under uncertainty [28]. They combine graph theory and probability theory to compactly represent joint probabilistic distributions using independence constraints [29]. PGMs have gained popularity due to their flexibility and improved learning and inference capabilities in large networks [30].
- **Case-Based Reasoning:** CBR systems solve new problems by referring to similar past cases stored in a knowledge base. They are commonly used in legal systems, customer support, and medical diagnosis. It has roots in cognitive science, machine learning, and knowledge-based systems [31]. Their strengths include these systems mimicking human problem-solving approaches. They are also efficient in domains with recurring problems. CBR typically involves four main processes: retrieval, reuse, revision, and retention of cases [32]. The methodology has been applied in various domains, including medicine, finance, and engineering [33]. For instance, a physician might use CBR to diagnose a patient by recalling a similar case from the past [33]. As good as CBR is, it faces several limitations. One major challenge is the heavy reliance on domain-specific knowledge, which reduces generality and adaptability [34]. Feature-based recognition in CBR systems struggles with identifying predictive features and recognizing structural patterns [35].

2.3 Logic Programming & Answer Set Programming (ASP)

2.3.1 Overview of Logic Programming & Its Role in Reasoning Tasks

Logic programming is a computational paradigm rooted in formal logic, primarily aimed at declarative problem-solving and reasoning. Unlike imperative programming, where algorithms are explicitly defined, logic programming focuses on defining relationships and rules that a system uses to infer conclusions. It is basically programming by description. The programmer describes the application area and lets the program choose specific operations. Logic programs are easier to create and enable machines to explain their results and actions [36]. Prolog, one of the earliest and most well-known logic programming languages, exemplifies this approach, enabling the encoding of complex relationships and querying them effectively [37].

At its core, logic programming operates on the principle of using facts, rules, and queries to deduce new knowledge. Facts represent basic truths about the domain, while rules encode logical relationships that allow the derivation of new information. Queries are posed to the system to retrieve specific insights or validate hypotheses. This approach is particularly effective in domains requiring knowledge representation, planning, and automated reasoning.

Logic programming plays a crucial role in reasoning tasks due to its ability to encode complex relationships, such as hierarchical structures and causal dependencies, in a formal yet human-readable syntax [38]. It excels in knowledge representation by translating real-world scenarios into structured, rule-based frameworks, providing a good foundation for computational analysis [39]. Furthermore, logic programming enables automated reasoning, allowing systems to derive implicit conclusions from explicitly stated facts and rules—a capability essential for applications like diagnostic systems, scheduling, and decision-making [40]. Critically, it also supports non-monotonic reasoning, where dynamically introduced information can override previous assumptions, making it adaptable to evolving contexts and real-world uncertainties [41].

2.3.2 Key Features & Advantages of Answer Set Programming (ASP)

Answer Set Programming (ASP) is an advanced form of logic programming tailored for declarative problem-solving. It extends the basic principles of logic programming by incorporating non-monotonic reasoning and stable model semantics, making it particularly suited for complex, knowledge-intensive domains [40]. ASP enables developers to express intricate constraints and rules, generating “answer sets” that represent solutions satisfying these constraints.

ASP distinguishes itself through several key features. Its declarative nature allows problems to be encoded as logical rules, with ASP solvers automatically deriving solutions that satisfy all constraints, bypassing the need for procedural algorithms [42]. Another strength lies in its expressive semantics, which support non-monotonic reasoning—enabling the representation of defaults, exceptions, and dynamic knowledge updates critical for real-world applications [43]. Practical implementations of ASP are further enhanced by efficient solvers like Clingo and DLV, which are optimized to handle large-scale problems while maintaining scalability and performance [44]. Finally, ASP benefits from inherent parallelism: its rule-based structure allows many problems to be decomposed and solved in distributed environments, significantly accelerating computation for complex tasks [44, 45]. These features collectively position ASP as a powerful tool for structured reasoning in dynamic and knowledge-intensive domains.

Key strengths of ASP in knowledge reasoning:

- **Flexibility:** ASP is highly adaptable, making it suitable for a wide range of problems such as planning, scheduling, configuration, and diagnosis. Its declarative nature allows users to describe a problem in terms of rules and constraints without specifying how to solve it. This versatility enables ASP to model diverse domains, from optimizing resource allocation in industrial processes to generating feasible schedules in logistics and education. Additionally, ASP’s ability to represent non-monotonic reasoning (where new information can revise earlier conclusions) allows it to adapt to changing requirements and contexts seamlessly.
- **Precision:** ASP provides a rigorous, formal method for defining and solving problems, ensuring correctness and consistency in reasoning. Its foundation in logic programming guarantees that every solution (or “answer set”) strictly adheres to the defined rules and constraints. This precision is especially valuable in critical applications such as medical diagnosis or legal reasoning, where small errors in reasoning could have significant consequences. Furthermore, the logical structure of ASP promotes transparency, enabling users to trace how solutions are derived, which is vital for debugging and validating complex systems.

- **Complex Decision-Making:** One of the strengths of ASP is its ability to handle problems with conflicting constraints or incomplete information. It can evaluate multiple possible solutions and identify those that best satisfy the defined rules, even when the problem space is vast or uncertain. This makes ASP particularly effective for real-world applications such as automated planning, where conflicts between resource availability and task deadlines are common, or in AI systems that need to reason about ambiguous or partially available data. ASP’s inherent support for reasoning about exceptions and priorities further enhances its capability to deal with complex decision-making scenarios.
- **Integration with Other Systems:** ASP can be integrated with other computational systems, such as machine learning models or natural language processing tools, to enhance its reasoning capabilities. For instance, machine learning algorithms can preprocess large, unstructured datasets to extract structured knowledge, which ASP can then use for logical reasoning. Similarly, in natural language understanding, ASP can work with language models to infer implicit meanings, relationships, or intentions from text. This hybrid approach combines the strengths of statistical learning and symbolic reasoning, enabling advanced applications such as intelligent assistants or automated knowledge graph construction.

2.3.3 Examples of Using ASP in Narrative Understanding and Decision-Making Systems

ASP has been successfully applied in domains requiring structured reasoning and decision-making, including narrative understanding. By representing events, intentions, and relationships as logical rules, ASP enables machines to reason about stories in a human-like manner.

In narrative understanding, ASP has been used to model stereotypical activities in restaurant settings, such as ordering food or paying bills, while inferring implicit details like a customer’s intent to dine or the causal link between ordering and receiving a meal [10]. For example, ASP rules can encode constraints like *“If a customer orders food, they expect service within a reasonable time”*, allowing systems to reason about deviations from expected norms, such as delays or missing actions.

ASP plays a pivotal role in interactive storytelling systems, where it guides narrative progression in games or adaptive stories by encoding plot constraints and character actions as logical rules [46]. This ensures narrative coherence while dynamically adapting to user choices, such as altering character decisions or plot outcomes without violating predefined story logic.

In healthcare, logistics, and robotics, ASP is widely employed for automated planning tasks such as resource allocation and scheduling. For instance, ASP can reason about optimal

task sequences under constraints like time, cost, and resource availability, enabling decision support systems to generate actionable plans for medical staff or autonomous robots [47].

ASP has been applied to model complex legal rules and ethical guidelines by encoding them as logical constraints. For example, systems can automatically assess whether a narrative describing a business’s actions complies with labor laws or ethical standards, leveraging ASP to infer violations or exceptions from encoded regulations [48, 49].

In forensic analysis and fraud detection, ASP has proven valuable for identifying inconsistencies in narratives, such as contradictory events or missing causal links. By formalizing narrative elements as logical predicates, ASP solvers can flag anomalies that indicate deception or errors, such as a character appearing in two places simultaneously or a payment occurring before a service is rendered [50].

2.4 Large Language Models (LLMs) in Natural Language Processing

2.4.1 Evolution of LLMs and Their Capabilities

The evolution of Large Language Models (LLMs) has been driven by advances in deep learning architectures and the increasing availability of computational resources. Early natural language processing (NLP) systems relied on rule-based methods or statistical models like n-grams and Hidden Markov Models (HMMs), which, while effective for basic tasks, lacked the capacity to capture nuanced context or semantic relationships. A significant shift occurred with the introduction of word embeddings such as Word2Vec [51] and GloVe, which represented words as dense vectors based on contextual usage. These models improved semantic understanding but remained limited to isolated word-level analysis, unable to process sentences or paragraphs as cohesive units. The breakthrough came with the Transformer architecture [1], which introduced self-attention mechanisms to model long-range dependencies in text, enabling large-scale pretraining on diverse corpora. This innovation paved the way for modern LLMs like GPT (Generative Pretrained Transformer) [52] and BERT (Bidirectional Encoder Representations from Transformers) [53]. GPT models pioneered autoregressive training for tasks like text generation, while BERT’s bidirectional approach revolutionized contextual understanding for applications such as sentiment analysis and question answering. Recent advancements have produced models like GPT-4 and T5, which scale to billions of parameters, support multilingual processing, and exhibit remarkable few-shot learning capabilities—performing complex tasks with minimal task-specific data. These developments have expanded LLMs’ utility across domains, from conversational agents to code understanding, while raising new challenges in areas like reasoning robustness and bias mitigation.

2.4.2 Specific Applications of LLMs in Extracting Structured Information

LLMs have demonstrated significant potential in extracting structured information from unstructured text, enabling diverse applications across domains. Table 2.1 summarizes key use cases:

Table 2.1: Specific Applications of LLMs in Structured Information Extraction

Application	Description	Example
Named Entity Recognition (NER)	Identifies entities (e.g., names, locations) in text and maps them to pre-defined categories [54].	Extracting participants and events from legal/historical documents.
Information Retrieval & Summarization	Generates concise summaries or structured reports by distilling key information from lengthy texts [55].	Creating symptom/diagnosis reports from clinical notes.
Event Extraction	Recognizes events, attributes, and relationships in narratives (e.g., news articles).	Extracting event type, participants, and location from news stories.
Knowledge Base Construction	Populates knowledge graphs by extracting entities and relationships from unstructured corpora [56].	Building a medical knowledge graph from research papers.
Dialogue Systems & Chatbots	Extracts user intents and slot-filling data for downstream task automation.	Booking systems parsing user queries like “Reserve a table for two tonight.”
Code Understanding	Extracts structured insights from code comments and documentation for software engineering tasks [57].	Generating API usage guidelines from developer notes.

2.4.3 Challenges LLMs Face in Narrative Understanding

Despite their advances in natural language processing, Large Language Models (LLMs) face significant challenges in narrative understanding due to the inherent complexity of stories. A primary issue is their limited capacity for **context retention** over long narratives. While Transformers theoretically handle long-range dependencies, fixed token windows in practice force LLMs to truncate or approximate extended contexts, leading to information loss critical for maintaining coherence in multi-paragraph stories [58]. This is exacerbated by the **structural complexity** of narratives, which often involve hierarchical event sequences (e.g., a main plot with subplots) that LLMs struggle to represent in their sequential processing framework. For example, a model might conflate primary and secondary events or fail to track nested timelines.

A second challenge lies in **reasoning limitations**. While LLMs excel at surface-level text generation, they often falter in multi-step reasoning tasks such as deducing implicit causal

relationships or character motivations. This is closely tied to their difficulty in handling **implicit information**—narratives frequently rely on cultural knowledge, tone, or contextual cues that are not explicitly stated [59]. For instance, an LLM might misinterpret sarcasm in dialogue or fail to infer that “John slammed the door” implies anger without explicit textual clues. These gaps hinder their ability to reconstruct the full narrative logic required for tasks like intention modeling or conflict resolution.

LLMs also exhibit **sensitivity to ambiguity** and **cultural biases**, both of which are pervasive in narratives. Ambiguous phrasing (e.g., “They saw her duck”) can lead to arbitrary interpretations, while biases from training data may skew narrative analyses toward dominant cultural perspectives [60]. For example, a story involving non-Western dining customs might be misrepresented if the LLM lacks exposure to diverse cultural contexts. Finally, integrating LLMs with formal reasoning frameworks like Answer Set Programming (ASP) remains nontrivial. While ASP requires structured inputs (e.g., predicates, rules) for logical inference, LLMs generate free-form text, necessitating error-prone translation steps that risk propagating inconsistencies into downstream reasoning tasks.

2.4.4 Overcoming the Challenges

Efforts to address the limitations of LLMs in narrative understanding focus on refining their ability to extract and structure information for downstream reasoning tasks. One prominent strategy is *prompt engineering*, where carefully designed prompts guide LLMs to generate outputs aligned with logic programming syntax, such as ASP predicates or temporal constraints [61]. For example, prompts may include templates like “*Identify events in the story as ASP predicates: event(type, actor, object)*” to enforce structured outputs. Another approach involves *fine-tuning LLMs* on domain-specific datasets annotated with narrative elements (e.g., causal relationships, intentions), which improves their ability to infer implicit context and adhere to formal syntax requirements. Hybrid frameworks that integrate LLMs with logic programming systems like Answer Set Programming (ASP) show particular promise—LLMs act as semantic parsers to extract structured data, while ASP performs deterministic reasoning on these inputs to infer intentions, resolve conflicts, or validate consistency [62]. Additionally, *memory-augmented architectures* incorporate external memory modules to help LLMs retain and reference contextual details across long narratives, mitigating token limitations and improving coherence in multi-step reasoning. These combined strategies aim to bridge the gap between statistical language understanding and symbolic reasoning, enabling more robust narrative interpretation systems.

2.5 Bridging LLMs with Logic Programming

Integrating Large Language Models (LLMs) with logic programming systems offers the potential to combine the strengths of statistical learning with structured reasoning. However, this integration faces significant conceptual and technical challenges, as well as opportunities for innovative hybrid solutions.

2.5.1 Conceptual Challenges and Technical Gaps

Integrating Large Language Models (LLMs) with logic programming frameworks presents several conceptual and technical challenges. One major challenge arises from the **differing paradigms** underlying these systems: LLMs rely on probabilistic reasoning to handle ambiguous or noisy inputs, whereas logic programming depends on deterministic rules and explicit facts, creating a fundamental mismatch in how they process information. This disconnect is exacerbated by a **representation mismatch**, where LLMs generate free-form natural language outputs, while logic systems like Answer Set Programming (ASP) require structured, syntax-bound inputs such as predicates (e.g., `event(order, John)`) or constraints (e.g., `:- not pay(X), order(X, Y)`). Translating between these representations remains nontrivial, often requiring manual intervention or error-prone post-processing. Additionally, LLMs exhibit **reasoning limitations** in performing multi-step logical deductions or resolving contradictions—tasks that are intrinsic to narrative understanding but demand the rigorous, rule-based reasoning native to logic programming. For instance, while an LLM might recognize that “John paid after ordering,” it struggles to infer implicit causal dependencies (e.g., `occurs_after(pay, order)`) without explicit training. Finally, **error propagation** poses a critical risk: inaccuracies in LLM-generated structured data (e.g., mislabeled entities or omitted events) can cascade through downstream logic-based reasoning, compromising the reliability of conclusions. These challenges collectively underscore the need for robust methods to align LLMs’ statistical strengths with the precision of symbolic reasoning.

2.5.2 Existing Attempts to Integrate LLMs with Formal Reasoning

Efforts to combine Large Language Models (LLMs) with formal reasoning frameworks have explored diverse methodologies to leverage their complementary strengths. One approach involves **semantic parsing**, where natural language narratives are translated into logical forms using fine-tuned LLMs. For example, systems like those proposed by [62] employ LLMs to parse text into ASP-compatible predicates (e.g., converting “John ordered coffee” to `event(order, john, coffee)`) through carefully engineered prompts or supervised training. Another strategy treats **LLMs as knowledge extractors**, where models like GPT-4 generate structured facts or rules from unstructured text, which are then processed by logic solvers for inference [63]. While promising, these methods often require manual post-processing to resolve ambiguities or syntax errors. Recent hybrid frameworks, such as those combining LLMs with Answer Set Programming (ASP) [63], demonstrate more robust integration. Here, LLMs preprocess narratives into structured predicates (e.g., temporal sequences like `occurs_after(pay, order)`), while ASP handles rule-based reasoning tasks like conflict resolution or intention modeling. However, these systems still face scalability challenges, particularly in domains requiring extensive commonsense knowledge or handling linguistically diverse inputs.

2.5.3 The Potential for Hybrid Models

Hybrid models combining Large Language Models (LLMs) and logic programming frameworks like Answer Set Programming (ASP) hold significant promise for advancing narrative understanding and reasoning. By integrating LLMs’ natural language processing capabilities with ASP’s structured logic, these systems can enhance narrative understanding through a two-stage pipeline: LLMs extract entities, events, and relationships from unstructured text (e.g., identifying `order(John, coffee)` from a story), while ASP applies domain-specific rules to infer implicit knowledge, such as causal dependencies or character motivations. This division of labor also improves explainability, as ASP’s deterministic reasoning provides transparent, auditable logic for decisions (e.g., flagging inconsistencies like unpaid orders), addressing the “black box” nature of purely statistical LLMs. Hybrid models also enable dynamic adaptation by allowing ASP to update its knowledge base with new information extracted by LLMs, supporting real-time adjustments in dynamic environments (e.g., evolving customer preferences in a service narrative). Finally, they support multi-modal integration, where LLMs process diverse inputs (text, images, or sensor data) into unified structured formats, enabling ASP to reason across modalities (e.g., correlating dialogue with visual scenes in interactive storytelling). These synergies enable hybrid systems to achieve robust, interpretable AI by merging the adaptability of statistical learning with the precision of symbolic reasoning.

Bridging LLMs and logic programming presents an exciting area of research in Computer Science, merging the capabilities of natural language understanding with structured reasoning for more robust, transparent, and adaptable systems.

2.6 Related Work: Comparative Studies and Gaps

Research at the intersection of Large Language Models (LLMs), logic programming, and narrative understanding has shown significant progress in recent years. This section reviews key contributions, identifies limitations in existing methodologies, and highlights gaps that the proposed research seeks to address.

2.6.1 Key Studies and Contributions:

- **Integrating LLMs with Logic Programming:** Recent work by Yang et al. has explored the use of LLMs combined with Answer Set Programming (ASP) to develop a system that can handle multiple question-answering tasks without the need for retraining for each new task [63]. Their method aimed to achieve state-of-the-art performance on various NLP benchmarks and robot planning tasks by leveraging the semantic parsing abilities of LLMs and the reasoning power of ASPs. Their method to achieve their objective in particular involved:
 - Using LLMs to parse natural language into atomic facts.
 - ASP modules would use these facts and encoded rules to perform reasoning tasks.

This system, according to the authors, achieved state-of-the-art performance on multiple NLP benchmarks, including:

- **bAbl** (QA and reasoning tasks).
- **StepGame** (multi-hop spatial reasoning).
- **CLUTRR** (inferring family relationships).
- **gSCAN** (compositional language and navigation tasks).

The main finding from this research was that this method outperformed baseline LLMs and prior neuro-symbolic approaches in accuracy and robustness.

The primary limitation of the proposed framework lies in the manual creation of ASP knowledge modules, which can be time-intensive and require domain expertise. While commonly used axioms, such as those for commonsense reasoning, can be reused, the vast amount of domain-specific and commonsense knowledge needed for broader applications remains challenging to encode. Additionally, the reliance on hand-crafted prompts to improve LLM performance limits scalability for more complex or diverse linguistic inputs. Furthermore, the framework was only tested using GPT models (e.g., GPT-3), and its performance with other large language models, such as Claude, PaLM, or LLaMA, was not explored. These models might yield different results, potentially revealing variations in reasoning ability, semantic parsing performance, or linguistic flexibility.

- **Generating Logic Programs with LLMs:** Ishay et al. have investigated techniques for using LLMs to generate Answer Set Programs directly from textual descriptions [64]. The goal of the paper is to propose a neuro-symbolic method that combines the strengths of large language models and answer set programming to transform natural language descriptions of logic puzzles into answer set programs.

A multi-step pipeline was developed where LLMs process logic puzzles in English to generate ASP rules in a structured manner:

- **Step 1:** Extract constants (e.g., entities and categories) using LLMs.
- **Step 2:** Generate predicates that define relationships among constants.
- **Step 2:** Generate ASP rules, including choice rules for generating possibilities (Generate phase) and constraints for eliminating invalid solutions (Define and Test phases).

Prompts are carefully engineered for each step, enabling LLMs to produce accurate and interpretable ASP programs. The pipeline was evaluated on datasets of logic puzzles, showing significant accuracy improvements compared to direct zero-shot and few-shot approaches with GPT-3 and GPT-4 [64]. The method generalizes well to other domains, such as Sudoku and various types of logic puzzles.

The proposed pipeline demonstrates significant potential in leveraging LLMs to generate Answer Set Programs (ASP) from natural language, but it has limitations such

as dependency on prompt engineering, occasional semantic and syntactic errors, and the need for human intervention to ensure correctness. Additionally, scalability to broader domains and transferability to diverse applications remain challenges due to the reliance on specific examples and prompts. Despite these limitations, the method effectively combines the linguistic strengths of LLMs with the reasoning capabilities of ASP, making formal logic programming more accessible and interpretable, and paving the way for future advancements in neuro-symbolic AI systems.

- **AutoConcierge:** Yang et al. in their research, introduce AutoConcierge, a conversational agent designed to provide restaurant recommendations by truly understanding user preferences through natural language interactions [65].

AutoConcierge distinguishes itself through three core capabilities. First, it integrates Large Language Models (LLMs) with Answer Set Programming (ASP): GPT-3 parses user dialogues into structured predicates (e.g., `prefers(user, cuisine=Italian)`), while the s(CASP) ASP system performs commonsense reasoning to interpret the semantics of user inputs, such as inferring unstated constraints like budget or dietary restrictions. Second, its interactive dialogue management proactively identifies missing information (e.g., a user specifying cuisine but omitting location) and engages users for clarification, ensuring recommendations align with nuanced preferences. Third, AutoConcierge operates as a domain-specific application focused on restaurant recommendations, enabling users to specify criteria like cuisine type, dining time, or group size. This specialization allows it to handle context-rich conversations while maintaining efficiency—a balance difficult to achieve in general-purpose systems.

The proposed system uses ASP to encode domain knowledge and dialog rules declaratively, Natural Language Processing (NLP) to preprocess and extract intents and entities from user input, and Dialog Management Framework to handle conversation flow based on inferred intents and knowledge-based reasoning.

The authors evaluated their framework on multiple domain-specific use cases, such as customer service and healthcare. The results demonstrated that the system could effectively interpret user dialogs, handle ambiguities, and provide accurate responses in a domain-specific context. Compared to traditional rule-based or ML-based dialog systems, the proposed approach showed improved reasoning capabilities and adaptability [65].

Its limitations include the system relying heavily on high-quality domain knowledge and rules, which may require significant effort to develop and maintain. The authors suggest exploring hybrid approaches combining declarative methods with machine learning for future work to improve scalability and generalization.

- **RestKB:** Inclezan et al. developed *RestKB* [66], a logic programming library that encapsulates commonsense knowledge for reasoning about restaurant narratives. RestKB models both routine activities (e.g., ordering meals) and exceptional scenarios like mistaken orders or interrupted dining experiences. Originally implemented in Answer Set Programming (ASP), it was reengineered using the Action Language for Modeling

(ALM) to improve modularity and testing capabilities. Key contributions include syntax extensions to ALM for non-deterministic action effects and flexible attribute constraints, enabling nuanced reasoning about narrative elements such as goals, events, and exceptions. RestKB adheres to knowledge representation principles like modularity and shallow inheritance hierarchies, making it adaptable to other stereotypical domains (e.g., medical visits). It also integrates with NLP tools to convert text into structured logic forms, providing a foundation for automated question-answering systems that infer implicit actions or diagnose narrative inconsistencies.

2.6.2 Limitations of Current Methodologies and Research Scope

While combining Large Language Models (LLMs) with logic programming (LP) frameworks like Answer Set Programming (ASP) for narrative understanding shows clear potential, research in this specific area is still emerging and faces several limitations regarding its current scope and depth. Much of the existing work has utilized relatively constrained narrative domains, such as specific scenarios or simplified logic puzzles. This focus raises significant questions about the **scalability** of these methods when applied to the longer, more ambiguous, and contextually rich narratives common in real-world applications, leaving their effectiveness on complex stories largely undemonstrated.

A critical gap, central to the motivation of this thesis, is the striking **lack of comparative studies** systematically evaluating how different foundational LLMs (e.g., models from the GPT family, Claude, Deepseek, etc.) perform *specifically* at the task of translating natural language narratives into structured, logic-compatible inputs. Current literature often employs a single model type without rigorous comparison, making it difficult to draw firm conclusions about which architectures or training paradigms might be most suitable or robust for extracting logical representations from diverse human stories.

Furthermore, the **generalizability** of the methods proposed so far remains largely unverified. Techniques involving prompt engineering or fine-tuning are frequently tailored to narrow datasets, and their portability to different narrative genres or subject matters is often untested. Integration strategies connecting LLM outputs to LP inputs tend to appear ad-hoc, lacking the standardized protocols needed for broader adoption or easier replication. Compounding these issues is a shortage of dedicated **benchmarks and standardized evaluation metrics** designed explicitly for this narrative-to-logic translation task, making objective assessment and comparison across different research efforts challenging. Finally, much of the focus has been on extracting surface-level facts, with less investigation into how these hybrid systems might tackle the deeper, more implicit reasoning often required for genuine narrative comprehension.

Overall, while foundational proofs-of-concept exist, extensive investigation is still required—particularly in systematically comparing LLM capabilities and testing approaches on more complex, varied narratives—to fully assess and realize the potential of integrating LLMs with logic programming for robust narrative understanding.

2.6.3 Research Gaps Addressed by This Work

The limitations outlined previously highlight specific, actionable gaps in the current understanding of how best to integrate LLMs with logic programming for narrative understanding. This thesis directly targets several of these gaps by employing a systematic methodology focused on comparison and optimization.

First, addressing the **critical lack of comparative LLM performance data** for this specific task, this research will systematically evaluate multiple contemporary Large Language Models. Their effectiveness in translating natural language narratives into structured Answer Set Programming (ASP) predicates will be quantitatively assessed using defined metrics such as precision, recall, F1-score for entity/event extraction, and crucially, syntactic validity percentage. This provides the empirical comparison currently missing in the literature, offering insights into model suitability for generating logic-compatible representations from diverse human stories.

Second, this work directly confronts the challenge of producing **reliable, logic-compatible outputs** from LLMs, moving beyond the often ad-hoc integration strategies seen previously. As outlined in the proposed methodology, this research will investigate and compare the efficacy of specific techniques aimed at enhancing output quality and ASP syntax adherence. This includes evaluating the impact of **domain-specific fine-tuning** using datasets pairing narratives with target ASP forms against sophisticated **prompt engineering strategies** utilizing structured templates and few-shot examples. The goal is to identify more robust methods for guiding LLMs to act as effective joint extractors and translators, directly generating structured data suitable for formal reasoning systems like ASP, minimizing the need for complex post-processing.

Furthermore, acknowledging the limited scope of many prior studies, this thesis incorporates an evaluation of **system generalizability**. By testing the developed framework and optimized LLM approaches on narratives from domains beyond the primary training data, this work provides insights into the cross-domain adaptability of using LLMs for structured narrative extraction—a key factor for practical application currently underexplored.

By systematically addressing these specific gaps—providing comparative LLM benchmarks for narrative-to-ASP translation, evaluating methods for optimizing logic-compatible output generation, and assessing cross-domain generalizability using defined metrics—this research aims to contribute concrete findings and refined methodologies. This work seeks to advance the practical integration of LLMs and logic programming, laying a more robust foundation for developing AI systems capable of deeper narrative understanding and reasoning.

Chapter 3

Methodology and Experimental Setup

This chapter details the experimental framework devised and implemented to investigate the central question of this thesis: how effectively can Large Language Models (LLMs) generate structured, ASP-compatible logic forms from unstructured narrative text? The research presented here establishes a concrete, reproducible experimental setup focused exclusively on in-context learning through varied prompting strategies. This design choice allowed for a controlled and systematic investigation into how different levels of guidance, namely, the number of example encodings provided and the availability of supplementary instructional material, affect the quality of LLM-generated logic forms.

The core of this experimental design is a comparative evaluation of multiple state-of-the-art LLMs on the narrative-to-ASP translation task. This chapter begins by defining the specific research questions that guided the empirical work, followed by a description of the dataset of 100 narratives curated for this study, including how it was partitioned into example encodings used within prompts and stories reserved for testing. The selection of the three LLM models is then justified, along with their configurations and the rationale for their inclusion.

A central component of this chapter is the articulation of the distinct experimental conditions. Each experiment varied along key dimensions: the number of worked examples of story-to-ASP translations provided in the prompt, and whether an additional instructional document covering edge cases, encoding conventions, and other helpful information was supplied alongside the prompt. This created a matrix of conditions designed to systematically assess the impact of these variables on output quality. The translation of these conditions into concrete prompt templates is fully specified, ensuring replicability.

Finally, the chapter outlines the experimental procedure itself, including the process for generating encodings across all 100 stories under each condition and model combination, as well as the sampling strategy employed to select a manageable subset for detailed analysis. The evaluation metrics and the manual annotation protocol used to assess the sampled outputs are deferred to the subsequent Chapter 4, where they are presented alongside the findings they produced. Together, this chapter establishes the methodological foundation upon which the empirical contributions of this thesis rest.

3.1 Research Questions

The experimental work presented here is guided by a refined set of research questions that focus specifically on the LLM’s generation task. These questions are:

LLM Narrative-to-Logic Translation Evaluation Methodology

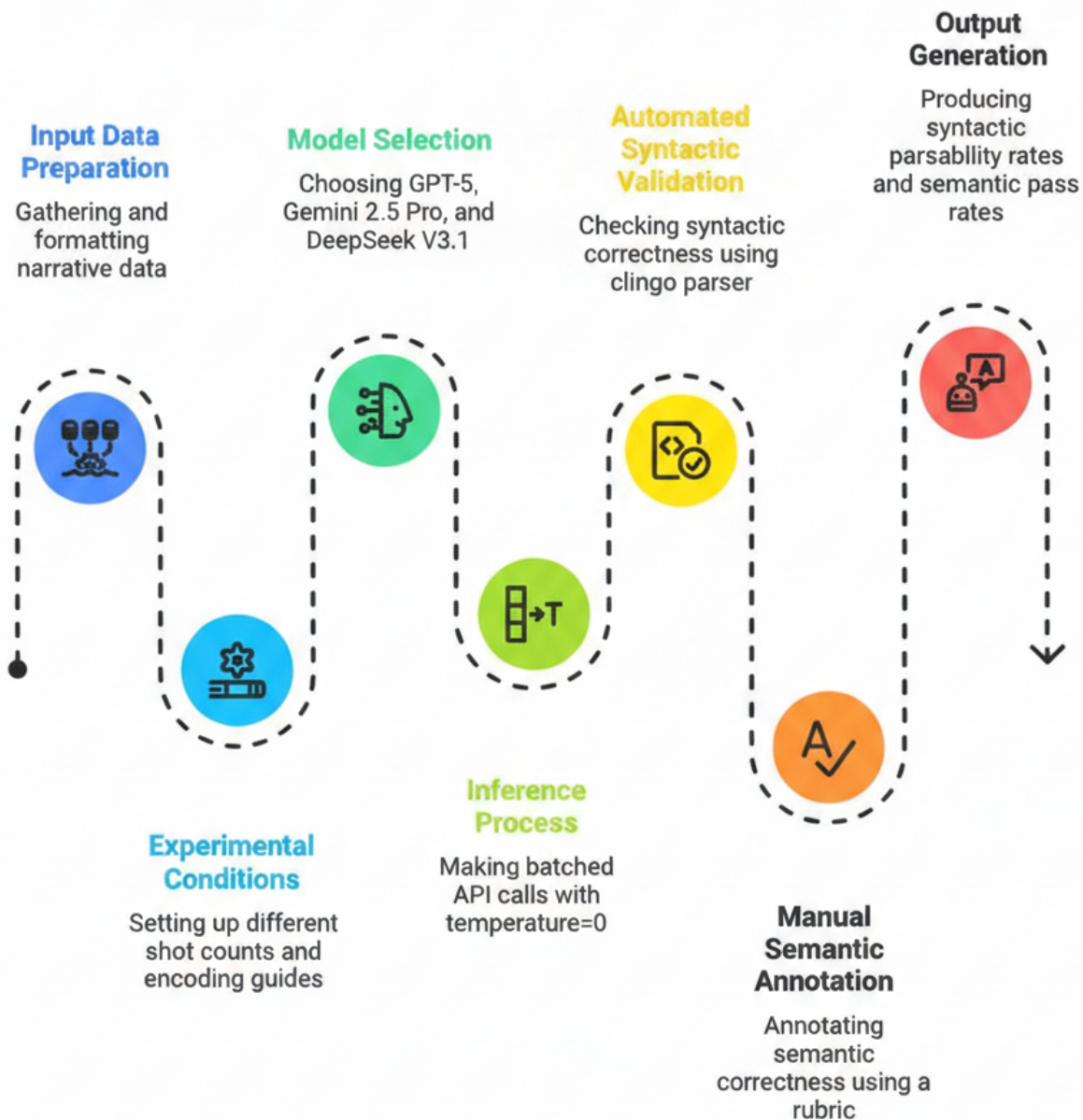


Figure 3.1: Overview of the experimental pipeline, illustrating the flow from narrative input through prompting conditions and model inference to the two-stage evaluation process.

RQ1: How can we design a structured pipeline that transforms narrative text into logic-compatible representations suitable for ASP?

This question addresses the architectural design of the experimental framework. It encompasses the construction of prompt templates, the integration of example encodings, the optional inclusion of supplementary instructional material, and the overall workflow from raw narrative input to structured ASP output.

RQ2: How do different LLMs compare in their ability to produce syntactically valid and semantically accurate ASP encodings under equivalent prompting conditions?

This question targets the comparative dimension of the study. By evaluating three distinct LLMs under identical experimental conditions, this research produces benchmark data on model-specific strengths and weaknesses for the narrative-to-ASP translation task, addressing the critical absence of head-to-head comparisons in the existing literature.

RQ3: What prompting strategies yield the most ASP-compatible narrative outputs from LLMs?

This question investigates how variations in prompt design, specifically, the number of worked examples provided and the inclusion of an explicit instructional document covering encoding conventions and edge cases, affect the syntactic validity and semantic faithfulness of the generated logic forms. The goal is to identify the most effective strategies for guiding LLMs toward producing usable ASP encodings.

RQ4: What are the most common structural and logical deficiencies in LLM-generated outputs from narrative text when intended as inputs for ASP logic programs?

Beyond aggregate quality metrics, this question probes the nature of the deficiencies in LLM outputs. By analyzing failure modes: syntactic errors, structural inconsistencies, and semantic misalignments across models and conditions, this study provides actionable insights into where current models falter, informing future efforts in prompt design and the broader integration of neural and symbolic approaches.

3.2 Data Preparation and Challenges

High-quality, annotated data is essential for benchmarking LLM performance in generating formal logic. This section outlines the complete lifecycle of the dataset construction used in this research. We detail the acquisition of natural language narratives, the methodology for manual extraction and segmentation, and the process of encoding these narratives into ground-truth ASP logic forms. Finally, we discuss the specific challenges encountered in mapping ambiguous human language to deterministic logic and how these were addressed.

Data Collection and Organization

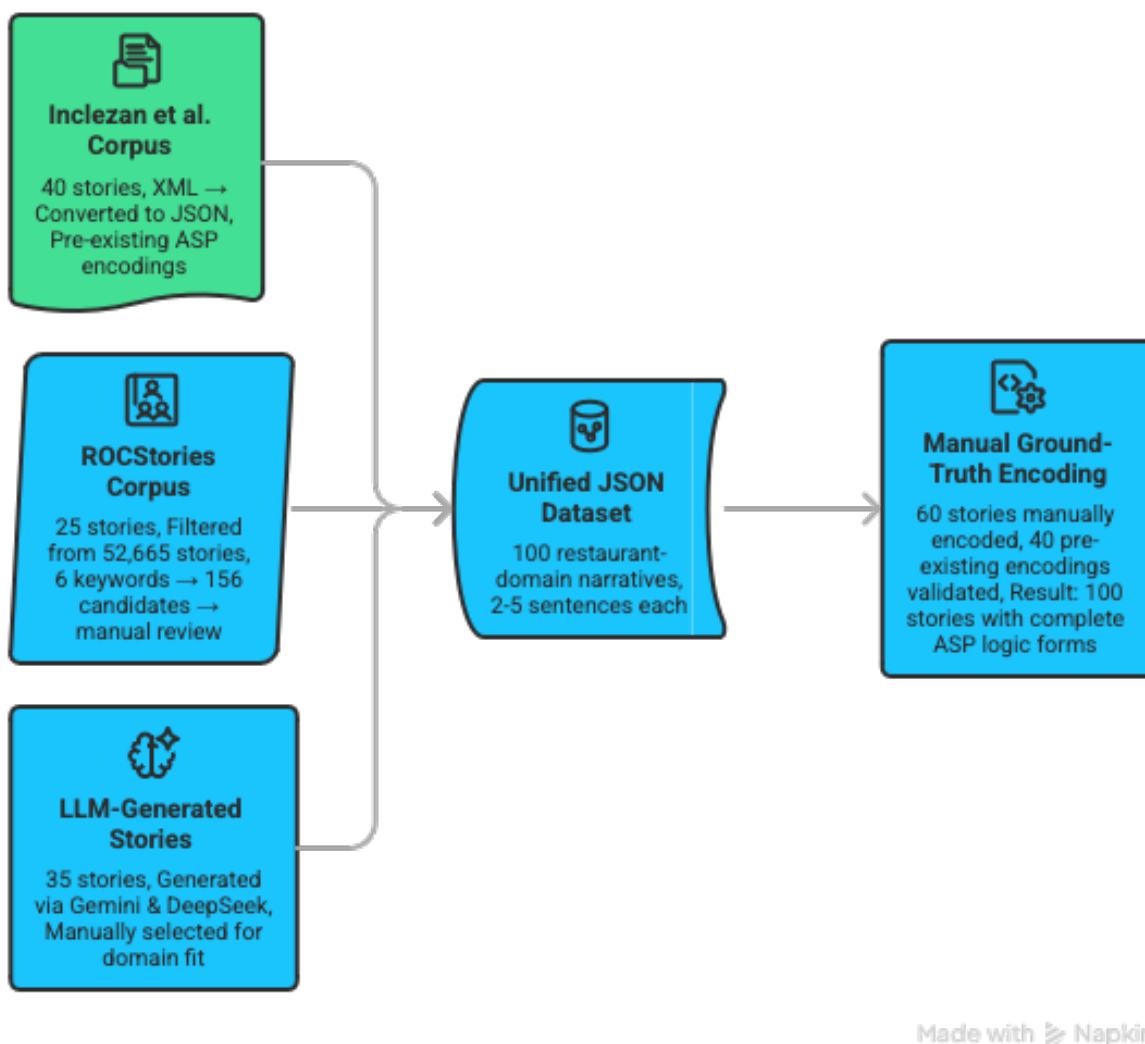


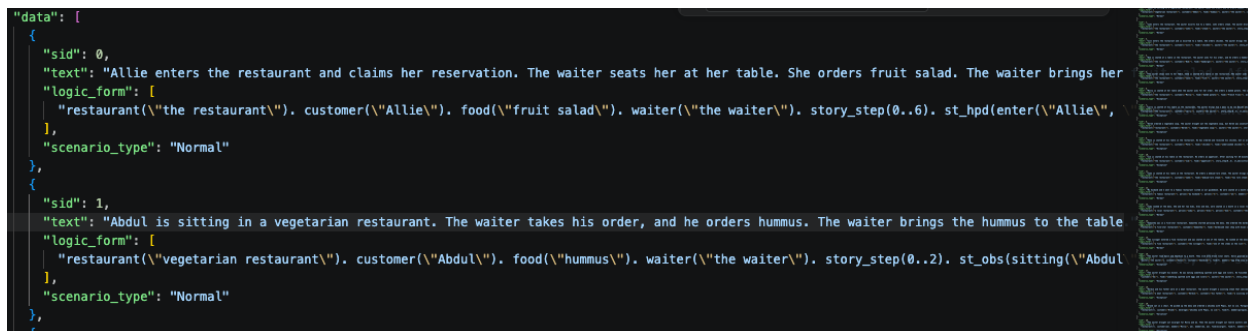
Figure 3.2: Overview of the data preparation pipeline. Three source corpora—Inclezan et al. (XML), ROCStories (CSV), and LLM-generated (plain text)—were processed and normalized into a unified JSON dataset of 100 restaurant-domain narratives, which then underwent manual ground-truth encoding to produce the final set of ASP logic forms.

3.2.1 Data Sourcing and Extraction

For this study, we curated a dataset of 100 natural language narratives focused on the restaurant dining domain. The majority of these narratives depict a typical dine-in scenario and range in length from two to five sentences. A subset of 40 stories was sourced from the existing corpus developed by Inclezan et al. [66], which was originally designed for ASP-based reasoning in restaurant contexts. To expand the dataset with the remaining 60 stories, we employed two distinct approaches.

First, we extracted narratives from the ROCStories corpus [67]. The ROCStories corpus was obtained via Amazon Mechanical Turk and contains collections of everyday stories, each consisting of four sentences followed by a possible ending sentence. We utilized a hybrid selection strategy combining automated keyword-based filtering with manual verification to ensure domain relevance. For the automated extraction, we experimented with various keyword combinations to identify terms most strongly associated with restaurant scenarios. We ultimately selected six keywords that provided an optimal balance between specificity and coverage: “diner,” “order,” “pizzeria,” “restaurant,” “steakhouse,” and “sushi.” Filtering the complete pool of 52,665 ROC stories using these keywords yielded 156 candidate narratives. Following a manual review to ensure alignment with our domain constraints, 25 of these candidates were selected for the final dataset.

The second approach, used to source the remaining 35 narratives, involved synthetic data generation using Large Language Models (LLMs), specifically Gemini 2.5 Pro and DeepSeek V3.1. To ensure the generated narratives were stylistically and structurally aligned with our domain requirements, we employed a few-shot prompting strategy. The models were provided with example stories from the Inclezan et al. [66] corpus to serve as templates. We observed that these LLM-generated narratives exhibited higher domain relevance and consistency with our specific restaurant scenarios compared to the general-purpose stories extracted from ROCStories. Although evaluating synthetic data generation is not the primary focus of this research, our experience suggests that using LLMs for targeted dataset augmentation is a highly viable strategy for obtaining structured narrative data.



```

"data": [
  {
    "sid": 0,
    "text": "Allie enters the restaurant and claims her reservation. The waiter seats her at her table. She orders fruit salad. The waiter brings her",
    "logic_form": [
      "restaurant(\\"the restaurant\\"), customer(\\"Allie\\"), food(\\"fruit salad\\"), waiter(\\"the waiter\\"), story_step(0..6), st_hpd(enter(\\"Allie\\",
    ],
    "scenario_type": "Normal"
  },
  {
    "sid": 1,
    "text": "Abdul is sitting in a vegetarian restaurant. The waiter takes his order, and he orders hummus. The waiter brings the hummus to the table",
    "logic_form": [
      "restaurant(\\"vegetarian restaurant\\"), customer(\\"Abdul\\"), food(\\"hummus\\"), waiter(\\"the waiter\\"), story_step(0..2), st_obs(sitting(\\"Abdul
    ],
    "scenario_type": "Normal"
  },
]

```

Figure 3.3: A representative sample of the unified JSON format for two stories, shown in the IDE.

The three subsets—Inclezan et al. (XML) [66], ROCStories (CSV) [67], and LLM-generated

(plain text)—were normalized into a unified JSON format for downstream processing. This involved extracting relevant fields from each source’s native structure and mapping them to a common schema containing the narrative text, a source identifier, and, where applicable, pre-existing ground-truth encodings. A representative sample of the unified JSON format, taken from the IDE, is shown in Figure 3.3.

3.2.2 Ground Truth Encoding

To establish a reliable baseline for evaluation, we ensured that all 100 narratives were paired with high-quality ground-truth ASP logic forms. For the 40 narratives sourced from the Inclezan et al. [66] corpus, we utilized the pre-existing, validated encodings. For the remaining 60 stories (sourced from ROCStories and generated by LLMs), we performed manual annotation adhering to the established ontological schema used in the referenced restaurant domain research.

Maintaining consistency with this specific ASP framework was critical, as the primary objective of this study is to assess the viability of LLMs in generating logic forms that are immediately usable by downstream ASP reasoning systems. Arbitrary encodings would render the output incompatible with the intended solver environments. Consequently, our manual encoding process enforced a set of structural requirements derived from the established schema. These included: using unary predicates (e.g., `restaurant/1`, `customer/1`, `waiter/1`, `food/1`) for entity declarations and `member/2` for group membership; mapping explicit actions to discrete time steps via `st_hpd(Action, true, Step)`; capturing contextual states through `st_obs(Fluent, Boolean, Step)`; and preserving narrative specificity by retaining proper names and dish descriptions as quoted strings. The complete predicate inventory including all entity types, fluents, actions, reserved constants, and special-case predicates such as `st_next/2`, is specified in Appendix A. This standardized schema ensured that all ground-truth encodings were consistent and compatible with the downstream ASP reasoning framework.

This standardization ensured that the dataset represented a cohesive testbed for evaluating the LLM’s ability to accurately extract explicit narrative elements and map them to the defined logical structure required by the downstream reasoning system.

3.2.3 Challenges in Dataset Construction

Dataset construction presented two primary challenges: sourcing representative narratives and mapping them to our strict ASP encoding format.

Schema Rigidity vs. Narrative Diversity A major difficulty stemmed from the inherent mismatch between the variability of human storytelling and the rigid set of predicates defined in our schema. We faced a significant trade-off during selection. If we only picked stories that perfectly matched our pre-defined predicates, we risked creating a dataset that was artificial and too specific, failing to test the model’s ability to handle diverse inputs.

On the other hand, including highly general stories often made encoding impossible without “inferring” implicit information, something we explicitly wanted to avoid to ensure the LLM focused on extraction rather than reasoning. We observed this trade-off clearly in our sources: LLM-generated stories were easier to encode because they aligned naturally with the domain, whereas narratives from ROCStories were more linguistically diverse but often lacked the specific details required by our logic forms. To balance this, we prioritized stories that contained explicit stereotypical events, ensuring valid encodings without relying on the model to guess unstated details.

Ontological Alignment and Role Compression The fixed set of available predicates required us to make specific ontological decisions during the manual encoding process. First, while the schema includes distinct predicates for restaurant staff roles such as, `waiter/1`, `host/1`, and `cook/1`, the `waiter/1` predicate serves as the default functional role for encoding service-related actions such as taking orders, serving food, and delivering the bill. Consequently, when a story mentioned a “host” seating a customer or a “cashier” handling payment, but did not mention the waiter explicitly, the service-related actions were still encoded using the constant `w` and the `waiter/1` predicate, as these fall within the functional scope of waiter duties in the target ASP theory, and other roles can not substitute for it. The `host/1` and `cook/1` predicates were reserved for characters explicitly identified as hosts or cooks who performed role-specific actions beyond standard service tasks. Second, certain aspects of a dining experience, such as a customer liking or disliking food or the timeliness of service, were excluded from the ground truth as the target ASP theory did not contain predicates to represent sentiment or relative latency. Third, we encountered ambiguity regarding the “entry” event. While sentences like “Joe went to the sushi place” map clearly to `enter(Joe, Restaurant)`, many narratives begin with the character already inside the restaurant (e.g., “Joe was sitting at the table”). To maintain consistency and avoid inferring an unstated entry action, we utilized the state fluent `in(Joe, Restaurant)` for the latter cases, reserving `enter/2` only for explicit transition events.

Action Granularity and Co-occurrence Finally, we established strict heuristics to handle action granularity, particularly regarding consumption. Narratives often describe an order containing both food and beverages but subsequently summarize the consumption with a single verb (e.g., “He ate his dinner”). To prevent inconsistency in the manual annotation, we adopted a rule of explicit inclusion: if the text only explicitly mentions `eat`, we do not encode a simultaneous `drink` action, even if a beverage was present in the order. This heuristic ensures that the ground truth reflects the text’s explicit surface form rather than the logical implications of the dining activity.

Temporal Segmentation and Simultaneity A further challenge arose in mapping narrative actions to discrete time steps. Our encoding schema requires each explicit action to be assigned a sequential integer timestamp. In most cases, the default heuristic of “one action per time step” applied cleanly: consecutive clauses in the narrative were mapped to

consecutive steps. However, certain scenarios admitted multiple valid temporal interpretations. For instance, when a waiter simultaneously places a dish on the table and informs the customer that the meal is complimentary, which would be actions corresponding to the predicates `put_down/3` and `inform/3`, it is equally defensible to assign these actions to the same time step (reflecting their co-occurrence) or to consecutive steps (reflecting a sequential reading). Rather than enforcing an arbitrary resolution rule that would privilege one interpretation, we adopted a permissive evaluation standard: during manual assessment, both temporal assignments were accepted as correct, provided they were otherwise semantically and syntactically valid. This decision acknowledges the inherent underspecification of natural language with respect to fine-grained temporal structure and ensures that LLMs are not penalized for making reasonable interpretive choices.

These encoding heuristics, while reductive, ensured that the ground truth annotations remained faithful to the surface content of the narratives, providing a consistent and defensible baseline against which the LLM-generated encodings could be evaluated.

3.3 Experimental Conditions and Prompt Design

With the dataset and encoding schema established, this section describes the experimental framework within which the LLMs were evaluated. We detail the selection of models, the design of distinct experimental conditions, and the structure of the prompts used to elicit ASP-compatible logic forms from narrative text. The experimental conditions were varied along two key dimensions: the number of worked examples of story-to-ASP translation pairs included in the prompt, and the presence or absence of a supplementary instructional document providing additional encoding guidance. All other variables, including the output expectations schema (Appendix A), the task instruction format, and the evaluation dataset were held constant across conditions to isolate the effects of these two factors.

The following subsections describe each component of the experimental design in turn.

3.3.1 Model Selection

All experiments were conducted via the OpenRouter API, a unified interface that provides access to a wide range of large language models from different vendors through a single endpoint. OpenRouter was selected for several practical advantages: it abstracts away vendor-specific authentication and formatting requirements, enables direct comparison across models under identical API conditions, and provides access to both proprietary and open-source models with consistent billing, rate-limiting behavior, and versioning metadata.¹

Three models were selected for the primary experimental conditions, each representing a distinct category within the contemporary LLM landscape. All three were chosen based on their status as stable, production-ready releases at the commencement of the experimental

¹Model release dates, version identifiers, and knowledge cutoff dates reported in this section are drawn from OpenRouter’s model metadata as of the experimental period (late 2025).

phase (late 2025), with preview or beta models deliberately excluded to ensure reliability and reproducibility.

GPT-5 (OpenAI). The OpenAI model, accessed via the identifier `openai/gpt-5` on OpenRouter (released August 7, 2025; knowledge cutoff September 2024), was selected as the representative proprietary model from the most widely used AI vendor globally [68]. As the current stable flagship release from OpenAI at the time of experimentation, it served as a baseline for state-of-the-art performance on structured generation tasks.

Gemini 2.5 Pro (Google). The Google model, accessed as `google/gemini-2.5-pro` (released June 2025; knowledge cutoff January 2025), was chosen as a second proprietary baseline. At the time of its release and continuing into the experimental period, Gemini 2.5 Pro was widely regarded as a leading model, particularly for natural language understanding tasks [69]. Its inclusion allowed for head-to-head comparison with the OpenAI offering under identical prompting conditions.

DeepSeek V3.1 (DeepSeek). The third model, accessed as `deepseek/deepseek-chat-v3.1` (released August 21, 2025; knowledge cutoff March 2025), represented the open-source dimension of the comparison [70]. Developed by a non-Western organization, DeepSeek V3.1 offered an opportunity to assess whether an open-weight model from a different research ecosystem could perform comparably to its proprietary Western counterparts on the niche task of narrative-to-ASP translation.

The selection of these specific versions was guided by a principle of temporal comparability: all three models were released within approximately three months of one another, reflecting a broadly contemporaneous generation of LLM capabilities. Although newer models became available during the experimental period—most notably a preview release from Google—these were excluded from the primary experiments to maintain a level playing field among stable, non-preview versions.

Table 3.1 provides an overview of all experimental conditions evaluated in this study.

Table 3.1: Summary of Experimental Conditions

Condition	Section	Examples	Output Expect.	Add. Info
ZeroShot-V1	3.3.3	0	Present	Present
FewShot-5-Random-V1	3.3.4	5	Present	Absent
FewShot-5-Random-V2	3.3.5	5	Present	Present
FewShot-10-Random-V1	3.3.6	10	Present	Absent
FewShot-10-Random-V2	3.3.7	10	Present	Present
Manual-Prompting	3.3.8	10	Present	Present

Supplementary Web-Based Trials. In addition to the API-based experiments, an exploratory comparison was conducted using the official web interfaces of the respective vendors. The model versions available through these interfaces at the time were GPT-5.1 [71],

Gemini 3.0 Pro, and DeepSeek V3.2.² Identical prompts from the API-based experiments were supplied through the web interfaces to assess whether the conversational versions of these models exhibited different behavior or output quality. These trials are treated as a supplementary data point and are reported separately from the primary experimental results.

3.3.2 Common Prompt Architecture

Before describing the individual experimental conditions, we first establish the prompt architecture that was shared across all conditions in this study. This common framework ensured that any differences in model output could be attributed to the specific variables under investigation rather than to incidental differences in prompt construction.

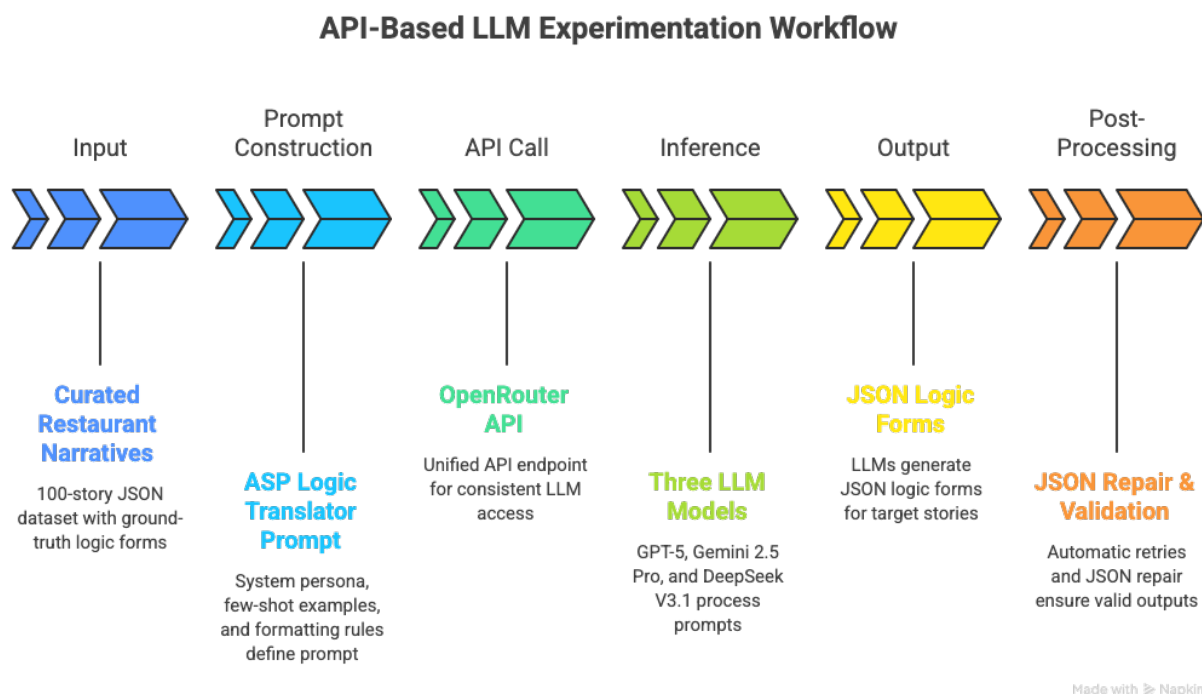


Figure 3.4: Overview of the experimental infrastructure, showing the flow from the input dataset through prompt construction and the OpenRouter API to the three primary models under consistent inference settings.

System Persona

Every API request across all conditions began with a static system persona assignment: “*You are a precise ASP logic translator. You never skip data.*” This instruction served two

²The version identifiers for the web-based trials reflect the models available through official vendor interfaces as of the trial date. These differ from the API versions used in the primary experiments.

purposes. First, it established the model’s role as a translator of natural language into a formal logical notation, signalling that the task required structured, rule-governed output rather than creative or conversational text. Second, the directive “*You never skip data*” was included as a countermeasure against the tendency of some models to omit or summarise inputs, a behavioural failure mode we discuss further in the Challenges section later in the chapter.

Instructional Documents

Every experimental condition included a shared instructional document located at `FewShot/docs/output_expectations.md`. This document defined the complete predicate inventory available for translation, including entity predicates (`customer/1`, `waiter/1`, `food/1`, `beverage/1`, etc.), the full set of fluents for state observations (via `st_obs/3`), and the complete inventory of actions for event occurrences (via `st_hpd/3`). It also specified critical formatting rules, most notably the requirement that all entity names be enclosed in double quotes, e.g., “John” or “miso soup” and defined reserved constants (`t`, `b`, `r`, `w`, `f`, `tip`) and the special predicate `st_next/2` for consecutive time steps in complaint scenarios. The full specification is reproduced in Appendix A. This document was present in all conditions, providing a consistent baseline specification across the entire experimental matrix. A second document, `FewShot/docs/additional_info.md`, was introduced as a variable across conditions. It was a supplementary encoding guide. It addressed specific encoding challenges not covered by the output expectations document, including the prohibition of inferring unmentioned actions, the correct format for negated actions (`st_hpd(Action, false)` without a time step), rules for compressing non-waiter roles into the `waiter/1` predicate, handling of unspecified food items, and a directive that `story_step/1` could be omitted from the encodings. The supplementary encoding guide is reproduced in Appendix B. The presence of this second document was one of the two independent variables manipulated across conditions (the other being the number of few-shot examples provided).

Structural Constraints and Formatting Rules

In addition to the system persona and instructional document, every prompt included a repeated set of structural constraints, defined in the code as `JSON_FORMATTING_RULES` and placed immediately before the task instructions:

CRITICAL RULES:

1. Return ONLY a valid JSON object.
2. The ‘logic_form’ field MUST be a list of strings.
3. ESCAPE quotes: `\\"person(\\\\"John\\")\\"`.
4. MANDATORY COMPLETENESS: You MUST process EVERY story provided in the ‘TARGET TASK’ section.
5. If I give you 10 stories, you must return exactly 10 logic forms.

These rules addressed the most common syntax failures observed during pilot testing. Rule 3 (quote escaping) was particularly important: JSON requires internal double quotes to be escaped with backslashes, but ASP predicates frequently contain quoted entity names (e.g., `person("John")`), creating a nested-quoting problem that models consistently struggled with. Rules 4 and 5 served as countermeasures against the tendency of some models to silently omit or summarise input stories, a failure mode we discuss further in the Challenges section later in the chapter.

Input Data Injection

Rather than paraphrasing or reformatting the story data, the complete input JSON structure was injected verbatim into the prompt. This design choice was primarily practical: the dataset had already been constructed in JSON format, and injecting it directly avoided the risk of introducing errors through reformatting. Additionally, presenting the target stories in the same JSON structure as the few-shot examples provided a consistent format for the model to follow when generating its outputs. The input was placed in a clearly delimited section of the prompt with a direct instruction to generate or replace the `logic_form` field for each entry.

Inference Configuration

All model queries across all conditions shared the following inference parameters:

- **Temperature:** 0.0, ensuring deterministic output and facilitating reproducibility.
- **Response format:** Enforced as a JSON object via the API parameter `{"type": "json_object"}`, which instructs the model to constrain its generation to valid JSON syntax.
- **Token limit:** Set to 32,000 tokens per request, sufficient to accommodate both the prompt and the expected output for a single batch.

Batch Processing Strategy

A practical constraint emerged from the models' output token limits: generating logic forms for all 90 or 95 target stories in a single API call would exceed the maximum output token allowance of every model tested. To address this, we adopted a batched inference strategy.

For each experiment, the input stories were first separated into two groups at the data level: **examples**, which already possessed populated `logic_form` fields and served as reference material, and **targets**, which were marked `["N/A"]` and required encoding. Only the targets were divided into batches. Each batch's API request then contained:

1. The system persona and task instructions.
2. The full set of reference examples (identical across all batches for a given experiment).

3. The output expectations document.
4. The supplementary encoding guide, if applicable
5. A subset of target stories, typically 10 per batch, for which the model was instructed to produce logic forms.

The prompt structure for each batch was as follows:

```

### REFERENCE EXAMPLES:
[example stories with populated logic forms]

### DOCUMENTATION:
[instructional document, if applicable]

### TARGET TASK:
STORIES TO PROCESS:
[batch-specific target stories with "N/A"]

```

After all batches for a given model completed, their outputs were reassembled in the original story order to produce the complete result set. Because each batch’s request included the full set of reference examples, the model had the same few-shot context available in every batch, ensuring consistency across the full dataset.

Retry and Error Handling

Each batch request was configured with up to two automatic retries upon failure. If all retries were exhausted, the raw model response (or the error message) was persisted to disk for diagnostic purposes. Additionally, a JSON repair layer using the `json-repair` library was applied to handle common formatting issues such as unescaped quotes within predicate strings, which occurred frequently given the nested quotation structure of the ASP predicates within JSON fields.

3.3.3 Condition: ZeroShot-V1

Motivation

The ZeroShot-V1 condition establishes the baseline for the study. In this configuration, the models were given no worked examples of story-to-ASP translations. Their only guidance was the two supplementary instructional documents. This condition answers the question: can LLMs produce valid ASP logic forms from narrative text using only a predicate specification, without any exemplar translations to learn from?

Input Data

The input file `FewShot/inputs/0_input.json` contained 100 stories, all of which were marked `["N/A"]` for their `logic_form` fields. Because there were no pre-populated examples, the batching strategy divided the full set of 100 target stories into batches of 10, with each batch receiving the two instructional documents as context but no reference translations.

Variable Parameters

Parameter	Value
Few-shot examples	0
Example selection strategy	N/A
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Present
Target stories	100
Batch size	10

3.3.4 Condition: FewShot-5-Random-V1

The **FewShot-5-Random-V1** condition represents the initial few-shot configuration employed in our study.

Objective

The objective of this condition was to determine whether providing five randomly selected exemplar translations, in conjunction with the output expectations document, would enable LLMs to generalize to the remaining unseen stories. This represents the minimal prompt augmentation beyond a baseline zero-shot configuration.

Input Data

The input file `FewShot/inputs/5_random_input.json` contained 100 stories, of which five possessed fully populated `logic_form` fields and 95 were marked `["N/A"]`. The five examples were selected at random from the pool of manually verified translations and covered a range of narrative structures:

- **sid 1 (Abdul, Normal):** A straightforward sequence of sitting, ordering, and receiving food.
- **sid 59 (Lucy, Normal):** A health-food visit involving two distinct order items, illustrating separate encoding of food and beverage predicates.
- **sid 78 (Betsy, Normal):** A diner scenario with a group food order using `member/2` and a tip payment.

- **sid 93 (Me and my mother, Exception):** An unavailability narrative requiring a negated fluent and a location transfer between restaurants.
- **sid 98 (Joe, Normal):** A hunger-to-satiety state change across time steps.

Four of these were **Normal**-type stories and one was an **Exception**-type story.

Variable Parameters

This condition was configured with the following experiment-specific parameters, which distinguish it from other conditions:

Parameter	Value
Few-shot examples	5
Example selection strategy	Random
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Absent
Target stories	95
Batch size	10

3.3.5 Condition: FewShot-5-Random-V2

Motivation

FewShot-5-Random-V2 was designed to isolate the effect of providing additional encoding guidance beyond the basic predicate inventory. Whereas V1 supplied only the output expectations document, V2 introduced a second supplementary document containing explicit heuristics for handling edge cases and encoding ambiguities.

Changes from V1

The sole difference from the V1 condition was the inclusion of a second instructional document, `FewShot/docs/additional_info.md`. This document, reproduced in Appendix B, addressed several specific encoding challenges that were not covered in the output expectations file:

- **Action inference rule:** If a story does not explicitly mention an action (e.g., entering, ordering, eating), the model must not infer it.
- **Negation format:** Negated actions must be encoded as `st_hpd(Action, false)` without a time step, distinguishing them from observed false fluents.
- **Role compression:** Characters who perform waiter duties but are not explicitly called “waiters” should be encoded as `waiter(X)`, unless a separate waiter is also mentioned, in which case they become `person(X)`.

- **Unspecified food items:** Generic terms like “food” or “meal” should be encoded as `food("food")`; if even a generic term is absent, the constant `f` should be used.
- **Inconsistency resolution:** Where the provided examples are inconsistent, the model should follow the encoding method that appears most frequently.
- **Optional `story_step()`:** The `story_step/1` predicate is not required. Including or omitting it has no effect on correctness.

Variable Parameters

Parameter	Value
Few-shot examples	5
Example selection strategy	Random
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Present
Target stories	95
Batch size	10

All other parameters—system persona, inference configuration, batch processing strategy, and retry logic—were identical to those described in Section 3.3.2. The input file and the set of five example stories were the same as in V1, ensuring that any differences in output quality could be attributed solely to the presence of the additional encoding guide.

3.3.6 Condition: FewShot-10-Random-V1

Motivation

This condition doubled the number of few-shot examples from five to ten while keeping the same documentation configuration as V1 (output expectations document only). The objective was to assess whether additional exemplars alone, without supplementary encoding guidance, would improve the models’ ability to generalise to unseen stories.

Changes from the Five-Shot Condition

The input file was changed to `FewShot/inputs/10_random_input.json`, which contained ten stories with populated logic forms and 90 marked `["N/A"]`. The ten examples provided broader coverage of both **Normal** and **Exception** scenario types and introduced additional encoding patterns not present in the five-shot set:

- **sid 5 (Normal):** A waiter-led seating sequence without a separate entry event, relying on `lead.to/3` from the initial state.

- **sid 6 (Exception):** An order substitution narrative (ordered baked potato, received French fries), demonstrating the encoding of expectation violations.
- **sid 7 (Exception):** A “decides not to order” story using `st_hpd(order(P), false)` for a negated action.
- **sid 15 (Normal):** A villager dining alone with explicit menu-reading (`pick_up/3`, `read_menu/1`) encoded via `menu_read/1`.
- **sid 35 (Exception):** An unavailability narrative involving a full restaurant, requiring `st_obs/3` with a negated fluent for table availability.
- **sid 43 (Normal):** A bar-seating scenario with a solo diner ordering a beverage, demonstrating the use of `drink/2` without a corresponding `eat/2`.
- **sid 50 (Exception):** A “ran out of soup” narrative requiring the `available/2` fluent with a subsequent re-order.
- **sid 59 (Normal):** A health-food visit with two distinct order items (food and beverage predicates encoded separately).
- **sid 74 (Normal):** A couple dining together, using the `member/2` predicate for group membership alongside individual eating and drinking actions.
- **sid 92 (Exception):** A first-person narrative (“I”) where the salad served was smaller than ordered, requiring an implicit expectation violation.

Three of these stories were also present as examples in the five-shot condition (sids 6, 50, and 59), while the remaining seven were new. The examples thus represented a more diverse set of narrative structures, including first-person perspective, group dining, and a broader range of exceptional scenarios.

Variable Parameters

Parameter	Value
Few-shot examples	10
Example selection strategy	Random
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Absent
Target stories	90
Batch size	10

All other parameters remained as specified in Section 3.3.2.

3.3.7 Condition: FewShot-10-Random-V2

Motivation

This condition replicated the documentation configuration of the five-shot V2 condition (both `output_expectations.md` and `additional_info.md` present) while maintaining the ten-example setup of the preceding condition. It serves as the 10-shot analogue of the V1-to-V2 transition.

Changes from the 10-Shot V1 Condition

Relative to FewShot-10-Random-V1, the sole difference was the inclusion of the `additional_info.md` document. The same encoding heuristics described in Section 3.3.5 (action inference prohibition, negation formatting, role compression, unspecified food handling, and inconsistency resolution) were appended to the prompt for every batch.

All other parameters such as, input file, example set, number of targets, batch size, and inference configuration remained identical to FewShot-10-Random-V1.

Variable Parameters

Parameter	Value
Few-shot examples	10
Example selection strategy	Random
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Present
Target stories	90
Batch size	10

3.3.8 Condition: Manual-Prompting (Web Interface, Latest Model Versions)

As noted in Section 3.3.1, an exploratory supplementary trial was conducted using the official web interfaces of each vendor rather than the OpenRouter API. This condition was motivated by the difficulties encountered during the API-based 10-shot experiments, where GPT-5 and Gemini 2.5 Pro frequently failed to produce complete output sets due to token limits and batch laziness. We wanted to determine whether the same limitations would persist when interacting with the models through their native web interfaces, where conversational context management and output handling differ from the programmatic API.

By the time these trials were conducted, the model versions available through the web interfaces had advanced beyond those being used via OpenRouter API during our primary experiments. The versions available were GPT-5.1, Gemini 3.0 Pro, and DeepSeek V3.2. While this meant the comparison was not perfectly controlled as newer models may perform

differently for reasons unrelated to the delivery mechanism, it also provided an opportunity to assess whether the latest releases had improved on the limitations observed in their predecessors.

Prompt Design

Unlike the API-based experiments, which used a structured batched prompt with separate sections for reference examples, documentation, and batch-specific targets, the web-based trial used a single continuous prompt that was pasted directly into the chat interface:

```
You are a precise ASP logic translator. You never skip data.
Translate all the narrative text provided in the json file
(10_random_input.json) to their ASP logic form. Out of 100 stories
in the input data, 10 of those stories will have their valid logic
form for you to get a reference from. Apart from those examples,
there will be two markdown files shared in this prompt, with one
file (output_expectations.md) covering the type of predicates that
we are looking for, and other important information, and the other
file (additional_info.md) covering other important information
regarding the translation from natural language stories to valid
ASP logic form. The latter file also contains some information
about possible edge cases.
```

For output, output should follow the same json format as input, but the fields like 'text', and 'scenario_type' can be discarded.

The input JSON file, the output expectations document, and the additional encoding guide were attached or pasted alongside this instruction. The prompt differed from the API-based equivalents in two notable ways. First, the output format requirement was relaxed: rather than reproducing the full input JSON structure (including `text`, `logic_form`, and `storyid` fields), the model was instructed to return only the `sid` and `logic_form` fields for each story. Second, there was no programmatic batching, the intent was for the model to translate all 100 stories in a single interaction.

In practice, however, the models handled this request differently. ChatGPT (GPT-5.1) proactively refused to generate all 100 outputs at once, responding that producing the entire translated JSON in one response “would exceed the model’s output limits and risks truncation or corruption of the result.” After several exchanges negotiating the approach, it was agreed to split the task into three chunks, `sid 0–33`, `sid 34–66`, and `sid 67–99` with each chunk requested sequentially through the conversation. This behaviour was unique to ChatGPT; both Gemini 3.0 Pro and DeepSeek V3.2 generated their full outputs in a single response without suggesting or requiring chunking.

Relationship to API-Based Conditions

This condition is most directly comparable to FewShot-10-Random-V2 (Section 3.3.6), as both used ten random examples and both supplementary documents. The key variables that differed were:

- **Model versions:** GPT-5.1, Gemini 3.0 Pro, and DeepSeek V3.2 (newer than the API-based primary models).
- **Delivery mechanism:** Web chat interface rather than OpenRouter API, with no automated batching or retry logic.
- **Output format:** The model was permitted to omit `text` and `scenario_type` fields from the output, simplifying the required JSON structure.
- **No automatic JSON repair:** Because the interaction was manual, any output malformations were handled on a case-by-case basis rather than through the automated `json-repair` pipeline.

Variable Parameters

Parameter	Value
Few-shot examples	10
Example selection strategy	Random
Instructional document (<code>output_expectations.md</code>)	Present
Additional encoding guide (<code>additional_info.md</code>)	Present
Target stories	100 (single pass, no batching)
Delivery method	Web chat interface
Models	GPT-5.1, Gemini 3.0 Pro, DeepSeek V3.2

3.3.9 Prompting Challenges

We encountered a number of challenges while prompting the models to produce the desired structured outputs. These fell into two broad categories: technical issues related to output generation limits and JSON formatting, and behavioral issues related to instruction following and context management.

Technical Challenges in Automated LLM Inference A primary challenge encountered during the initial phase of experimentation was the discrepancy between the models’ input context windows and their maximum generation limits. While state-of-the-art models such as DeepSeek V3 and Gemini 2.5 Pro boast context windows exceeding 128,000 tokens, their output generation is governed by a separate `max_tokens` parameter. In our early experimental runs, after trying out varying number of tokens this parameter was set to 6,000

tokens. We chose this number because it worked well for the first experiments of zero shot and five shot configurations. However, when we started experimenting with the 10-shot prompts under this configuration, the resulting JSON structures, each containing ASP logic forms for multiple stories, frequently exceeded the limit, leading to mid-response truncation and **Unterminated String** errors. This behaviour from these two models initially forced us to substitute these models with their lightweight counterparts—GPT 4o and Gemini 2.5 flash—for the 10-shot experiments.

It came to our attention late in the research timeline that several LLM providers had relaxed their maximum output token allowances. We subsequently increased the `max_tokens` parameter from 6,000 to 32,000 and re-ran the affected experiments. Under this revised configuration, the truncation behavior disappeared entirely: all three primary models were able to produce complete encodings for every story supplied as input, without the mid-response failures observed previously. This confirmed that the earlier truncation was not an inherent limitation of the models themselves, but rather a consequence of a restrictive API parameter setting. All results reported in Chapter 4 are drawn from the final experimental runs conducted with the 32,000-token configuration.

A secondary technical challenge was the syntactic fragility of JSON. Models frequently failed to correctly escape internal double quotes within logic predicates, for example, writing `member("John", g)` instead of the JSON-compliant `member(\"John\", g)`. This necessitated the integration of a specialized JSON repair layer (using the `json-repair` library) to ensure the syntactic integrity of the generated datasets without losing valid data points to minor formatting errors.

Behavioral Inconsistencies and Context Management Beyond the token-limit issue, we observed one additional challenge related to context management. Because API-based LLM inference is stateless, the models do not retain memory of previously processed batches. This required global context injection, where few-shot examples had to be re-introduced into the prompt of every batch to ensure consistent logical encoding across all 100 stories. Without this measure, outputs from later batches would drift from the encoding conventions established in the earlier batches, as the model had no persistent memory of the examples it had previously seen. This batch-level re-injection of examples was applied consistently across all experimental conditions.

Chapter 4

Results and Evaluation

Having established the experimental framework, including the dataset, prompting conditions, and model configurations in the preceding chapter, this chapter presents a systematic evaluation of the Large Language Models’ ability to generate syntactically valid and semantically accurate Answer Set Programming (ASP) encodings from unstructured narrative text. Where Chapter 3 detailed *how* the data were generated, this chapter addresses the central empirical question: *how well* did the models perform?

The evaluation is structured to directly address the research questions posed in Section 3.1. Specifically, we assess RQ2: the comparative performance of GPT-5, Gemini 2.5 Pro, and DeepSeek V3.1 under equivalent conditions, RQ3: the effectiveness of different prompting strategies (varying shot counts and the presence of supplementary instructional documents), and RQ4: the common structural and logical deficiencies observed in the generated outputs. The overarching goal is to determine whether a reliable pipeline (RQ1) can be constructed to transform natural language into logic-compatible representations.

4.1 Evaluation Metrics and Annotation Protocol

To enable a reproducible and fine-grained assessment, we employed a two-stage evaluation protocol. The first stage measured *syntactic validity* by determining whether a generated ASP encoding could be successfully parsed by the clingo grounder. The second stage assessed *semantic accuracy* for outputs that passed the syntactic filter, comparing their logical content against the ground-truth encodings through manual annotation.

4.1.1 Syntactic Validity: Automated Clingo Parsing

Syntactic evaluation focused exclusively on whether generated ASP encodings would be parsable by clingo, which is one of the standard grounder and solver for Answer Set Programming. A generated logic form was considered syntactically valid if and only if it could be successfully parsed by the clingo grounder without raising a lexer or syntax error.

To operationalize this criterion at scale, we developed an automated validation pipeline using the official clingo Python library (version 5.8). The process proceeded as follows:

1. **Extraction:** For each experimental condition and each model, the generated JSON outputs were parsed to extract the predicted ASP predicate strings from the `logic_form` field.

2. **Conversion:** Each extracted predicate string was converted from its JSON representation to a flat string format suitable for clingo ingestion.
3. **Parsing:** The predicate string was fed directly to the clingo parser via the `Control.add()` and `Control.ground()` methods. These methods invoke the grounder’s lexer and parser without performing any solving.
4. **Classification:** If the parser raised an exception (indicating a lexer error, syntax error, or malformed predicate), the encoding was marked as *syntactically invalid*. If no exception was raised, the encoding was marked as *syntactically valid*.

This automated validation was applied exhaustively to *all* generated encodings across every experimental condition, model, and story. The exhaustive nature of this evaluation, covering all 100 stories across all conditions, provides a comprehensive and reproducible baseline for comparing model performance on syntactic correctness.

4.1.2 Semantic Accuracy: Manual Annotation Protocol

While syntactic validation could be automated exhaustively using the clingo parsing pipeline described in Section 4.1.1, semantic accuracy presented a fundamentally different challenge. Natural language narratives admit multiple valid interpretations, and the mapping from story text to ASP encoding is inherently underspecified. As established in Section 3.2.3, two encodings that differ—for instance, in whether co-occurring actions are assigned the same time step or consecutive steps—may both be semantically correct. Automating semantic evaluation would require encoding a theory of permissible variation, which is impractical given the nuanced ways in which logically equivalent encodings can differ.

Consider the following example, drawn from the evaluation set (SID 29, predicted outputs from two different model runs). The ground-truth encoding for this story placed the **informed** action at time step 2, concurrent with **put_down**. One model’s prediction followed this assignment, while another placed **informed** at time step 3, after the serving action:

Story: "Nicole went to a vegetarian restaurant. She ordered lentil soup. When the waitress brought her the soup, she told Nicole that it was on the house. Nicole enjoyed the soup and then left."

Predicted A (timestep 2):

```
... st_hpd(order(...), true, 1).
st_hpd(put_down("the waitress", "lentil soup", t), true, 2).
st_obs(informed("the waitress", "on the house", "Nicole"), true, 2).
st_hpd(eat(...), true, 3). ...
```

Predicted B (timestep 3):

```
... st_hpd(put_down("the waitress", "lentil soup", t), true, 2).
st_obs(informed("the waitress", "on the house", "Nicole"), true, 3). ...
```

Both encodings are semantically defensible: the waitress may plausibly inform the customer *while* placing the dish (same time step) or *immediately after* (consecutive step). A purely automated evaluation that enforced exact timestamp matching would incorrectly penalize the second encoding. Moreover, certain scenarios admit multiple valid predicate choices. For example, `st_hpd(pick_up("customer", m, t), boolean, timestep)` and `st_hpd(read_menu("customer"), boolean, timestep)` may both be correct for a narrative describing a customer examining a menu, and encoding both actions sequentially is also acceptable.

Given these considerations, such as multiple valid timestep assignments, alternative predicate selections, and the general underspecification of natural language with respect to logical form, we determined that manual annotation by human evaluators was the most appropriate method for assessing semantic accuracy.

Sampling Strategy

Exhaustive manual evaluation of all generated encodings was logistically infeasible. Across five API-based experimental conditions, three primary models, and approximately 100 target stories per condition (excluding example stories provided in few-shot prompts), the total number of encoding exceeded 1,500. Adding web-interface based supplementary experiment would push this number beyond 1800. A complete manual review at this scale would be prohibitively time-consuming and prone to annotator fatigue, which could compromise consistency.

We therefore adopted a stratified sampling strategy designed to capture performance variation across the full spectrum of encoding quality. Rather than relying on simple random sampling, which might over-represent easy cases or under-represent edge cases, we used an automated pre-filtering step to select stories representing a range of difficulty levels.

The pre-filtering process worked as follows. We ran a Python script on the outputs of a single model-condition combination—DeepSeek V3.1 in the FewShot-10-Random-V1 condition—that computed a coarse similarity score between each predicted encoding and its corresponding ground truth. The score was defined as:

$$\text{Similarity} = \frac{\text{Number of correctly predicted predicates}}{\text{Total number of predicates in ground truth}}$$

This scoring mechanism was intentionally coarse; it served only to rank-order stories by approximate alignment with the ground truth, not to provide a definitive semantic judgment. The script’s output was used to bin stories into approximate quality tiers.

The choice of DeepSeek V3.1 under the FewShot-10-Random-V1 condition was pragmatic. At the time of sampling, no manual annotations had yet been performed, so we had no basis for selecting one model-condition combination over another based on semantic quality. We assumed the 10-shot condition would represent a middle-of-the-road level of difficulty. Furthermore, due to the output token limit issues described in Section 3.3.9, GPT-5 and Gemini 2.5 Pro had not yet produced complete results for this condition, whereas DeepSeek V3.1 had. DeepSeek was therefore the only primary model for which a full set of outputs

was available at that stage, making it the natural choice for the sampling procedure.

From the resulting tiers, we selected 30 stories to ensure coverage across the full spectrum of similarity scores. Specifically, we sampled approximately three stories from each decile bin (e.g., 0.0–0.09, 0.1–0.19, . . . , 0.9–1.0). This stratified approach ensured that our manual annotation sample included examples of complete failures (low similarity), partial successes (mid-range similarity), and near-perfect or perfect encodings (high similarity), providing a representative picture of model performance across the difficulty spectrum.

These same 30 stories were then evaluated across all experimental conditions and all models. Using a fixed set of stories ensured that we were comparing model outputs on identical inputs, an apples-to-apples comparison, and eliminated story-level variability as a confounding factor in our semantic evaluation. The 30 stories were annotated using the 0–5 rubric defined in Table 4.1.

Annotation Rubric

We developed a scoring rubric from 0 to 5 to evaluate semantic accuracy, with an additional special category (4.5) for encodings that surpassed the ground truth. Each encoding was assigned a single score based on holistic judgment of its correspondence to the ground truth and the narrative text. The rubric is defined as follows:

Table 4.1: Semantic accuracy annotation rubric

Score	Description
0	The model did not produce any encoding for the story.
1	The model did produce encodings, but almost everything is wrong, or it completely hallucinates.
2	Some predicates are correct, but there are two or more clear mistakes.
3	Close to being considered correct, but contains one or two major mistakes (e.g., inferring an action not explicitly mentioned, making up a predicate, getting the timestep wrong for an action or a fluent, etc.).
4	Considered correct and passes the test. May not be perfectly encoded—acceptable minor variations include encoding <code>r</code> instead of <code>"restaurant"</code> , <code>w</code> instead of <code>"waiter"</code> , or omitting an <code>enter/3</code> action when the story is not super explicit (e.g., “Ana visited a restaurant by the river...”).
4.5	Special category: the LLM-predicted encoding appears to be <i>better</i> or more logically coherent than the manually prepared ground truth. (This occurred with surprising frequency.)
5	Exact match with the ground truth.

Scores of 0, 1, 2, or 3 were classified as *failures* (semantically inaccurate), while scores of

4, 4.5, or 5 were classified as *passes* (semantically accurate). The threshold at 4 reflects our judgment that minor, semantically inconsequential variations should not penalize a model.

Annotation Procedure

All manual annotations were performed collaboratively by the author and Dr. Daniela Incezan. For the first experimentally evaluated condition, a sample of 30 story encodings was scored independently by both annotators using the rubric in Table 4.1. Inter-annotator agreement exceeded 90%, indicating substantial consistency in applying the scoring criteria. Based on this high level of agreement, it was determined that the author would proceed with scoring the remaining encodings independently. However, for ambiguous or borderline cases encountered during the annotation process, the author regularly consulted with Dr. Incezan, and any disagreements were resolved through discussion to reach a consensus score.

The following sections present the semantic evaluation results for each experimental condition, drawing on this manually annotated sample. Where relevant, we also include qualitative examples that illustrate the observed success and failure patterns.

4.2 Syntactic Evaluation Results

Applying the automated clingo parsing pipeline described in Section 4.1.1, we evaluated the syntactic parsability of all generated ASP encodings across the five API-based experimental conditions. This section presents the aggregate results for each condition, followed by a detailed analysis of the specific failure modes observed.

4.2.1 ZeroShot-V1

The zero-shot condition, in which models received no exemplar translations and only the supplementary instructional documents, establishes the baseline syntactic competence of each model.

Table 4.2: ZeroShot-V1 syntactic parsability

Model	Total	Parsable	Not Parsable	Parsability Rate
ChatGPT 5.0	100	80	20	80.0%
DeepSeek V3.1	100	99	1	99.0%
Gemini 2.5 Pro	100	80	20	80.0%

DeepSeek V3.1 achieved near-perfect syntactic parsability with only a single failure, while both ChatGPT 5.0 and Gemini 2.5 Pro produced invalid output for 20 of their 100 encodings.

Failure Analysis

All 20 invalid outputs from ChatGPT and all 20 from Gemini share a single, uniform cause: the absence of terminating periods. In the ASP language, every fact or rule must end with a period (.). However, both models consistently generated facts separated only by whitespace, producing strings such as:

```
person("Jasper") person("Frank") customer("Jasper") customer("Frank")
restaurant(r) waiter(w) food("roast beef") st_hpd(pay("Frank", b), true, 2)
```

This format resembles a space-delimited token sequence rather than a valid clingo program. Crucially, the error pattern is uniform across all 40 invalid outputs: every predicate in every failing encoding is individually well-formed—correct arity, properly quoted strings, balanced parentheses—yet the absence of a single . character after each predicate renders the entire program unparsable. This indicates that both ChatGPT and Gemini, when operating without exemplars, default to a serialisation convention that is incompatible with clingo’s grammar.

DeepSeek’s single failure (SID 71) was qualitatively different: a stray closing parenthesis inserted mid-argument:

```
st_hpd(put_down("waitress", "glass of water"), t), true, 5)
```

The extra) after "glass of water" creates a malformed term structure that the parser cannot resolve. This is an isolated punctuation error rather than a systematic formatting issue.

4.2.2 FewShot-5-Random-V1

The introduction of five worked examples produced a sharp divergence in syntactic outcomes across the three models.

Table 4.3: FewShot-5-Random-V1 syntactic parsability

Model	Total	Parsable	Not Parsable	Parsability Rate
ChatGPT 5.0	95	95	0	100.0%
DeepSeek V3.1	95	74	24	77.9%
Gemini 2.5 Pro	95	95	0	100.0%

ChatGPT and Gemini both achieved perfect syntactic parsability, demonstrating that a small set of exemplars was sufficient to teach them the period-delimited formatting convention. DeepSeek, however, exhibited the opposite trend: its parsability rate dropped from 99% in the zero-shot condition to 77.9%.

Failure Analysis

The 24 DeepSeek failures in this condition fall into two categories, both distinct from the zero-shot error pattern.

Unsafe free variables (19 of 21 failures). The dominant failure mode—accounting for nearly all invalid outputs—is the use of free variables as placeholders for unspecified entities. When a story describes a character ordering “food” or “a meal” without naming the specific dish, DeepSeek’s encoding substitutes a variable rather than using the constant `f` (the convention established in the ground-truth encodings and the instructional documents). For example:

```
st_hpd(order(they, X, w), true, 1). st_hpd(eat(they, X), true, 2).
```

In clingo’s safety requirement, every variable appearing in a rule or fact must appear in at least one positive predicate in the rule’s body, ensuring that the variable can be grounded to a finite domain. Facts containing variables, such as the examples above, have no body predicates to ground `X`, making the variable unsafe. The parser rejects such constructs with an “unsafe variables” error.

This error appears across a diverse range of narrative structures. DeepSeek uses `X` for unspecified food items (`order(they, X, w)`, `eat("Tim", X)`), `Y` for unspecified table locations (`put_down("the waiter", Y, t)`), and occasionally both in a single encoding:

```
restaurant("a restaurant"). customer("Tim"). waiter("the waiter").
story_step(0..3). st_hpd(eat("Tim", X), true, 0).
st_hpd(put_down("the waiter", "their food", t), true, 1).
st_hpd(informed("Tim", "something was wrong", "the waiter"), true, 2).
st_hpd(put_down("the waiter", Y, t), true, 3).
```

The pattern suggests that DeepSeek treats the predicate vocabulary as a template into which slot values must be inserted. When a slot (e.g., the specific food ordered) is not explicitly filled by the story text, the model produces a variable rather than falling back to a default constant. This is in contrast to ChatGPT and Gemini, which in the same situation use either a generic constant or omit the predicate altogether.

Missing output (2 of 21 failures). Two target stories (SIDs 36 and 44) had no valid predicted output, one was missing entirely from the model’s response and the other contained a null value. These completeness failures are distinct from the syntactic errors described above.

4.2.3 FewShot-5-Random-V2

The addition of the `additional_info.md` encoding guide—which included explicit heuristics for handling unspecified entities, negated actions, and role compression—yielded a partial improvement in DeepSeek’s syntactic performance while introducing a new and unexpected failure mode.

Table 4.4: FewShot-5-Random-V2 syntactic parsability

Model	Total	Parsable	Not Parsable	Parsability Rate
ChatGPT 5.0	95	95	0	100.0%
DeepSeek V3.1	95	80	15	84.2%
Gemini 2.5 Pro	95	95	0	100.0%

ChatGPT and Gemini maintained their perfect scores. DeepSeek improved from 76.6% to 83.5%, suggesting that the supplementary heuristics partly addressed the unsafe-variable problem. However, the nature of DeepSeek’s remaining failures changed dramatically.

Failure Analysis

While the number of unsafe-variable errors dropped from 21 to 2, a new failure mode emerged that accounts for 12 of the 15 invalid outputs: character corruption in the form of random Chinese character insertions.

The corrupting character is consistently the Chinese character 极 meaning “extreme” or “extremely” (pronounced *jí*). Its appearance follows no discernible semantic pattern. It is inserted:

- **Within predicate names:** the character splits the predicate token, producing an unrecognised identifier (e.g., `st极_hpd(drink(...))`).
- **Within string literals:** embedded inside a quoted entity name (e.g., `waiter("极the waiter")`).
- **Adjacent to numbers:** corrupting time-step arguments (e.g., `true, 极4`) or `true, 7极`).
- **Adjacent to variables:** attached to constant or variable tokens (e.g., `f极` or `put_down(w极, ...)`).
- **Within other tokens:** producing a hybrid of a truncated predicate name and the corrupting character (e.g., `st极pd(...)`).

The following example, taken from SID 67, illustrates the severity of the corruption:

```
restaurant("a quiet Italian restaurant"). customer("Maria"). customer("Leo").
beverage("a bottle of red wine"). food("two pasta dishes").
waiter("[jí]the waiter"). st_hpd(lead_to("the waiter", "Maria", t), true, 1).
st_hpd(order("Maria", "a bottle of red wine", "the waiter"), true, 2).
st_hpd(drink("Maria", "a bottle of red wine"), true, 7). st[jí]_hpd(drink("Leo",
"a bottle of red wine"), true, 7[jí]). st_hpd(pay("Maria", b), true, 8).
```

Note the three distinct corruptions in this single encoding: a prefixed 极 inside a string literal, an infixed 极 splitting a predicate name, and a suffixed 极 adjacent to a numeric argument. Each of these is independently sufficient to cause a parse failure.

The mechanism underlying this corruption appears to be a token-level hallucination. During generation, the model’s output distribution assigns non-zero probability to a token that is semantically unrelated to the ASP domain. The token 极 corresponds to a single Unicode character that, in DeepSeek’s tokeniser, may be represented as a standalone byte or token. Its repeated appearance across multiple independent outputs (13 of 15 failures in this condition) suggests that the corruption is not a random sampling artifact but rather a systematic failure mode triggered by the interaction between the supplementary encoding guide and DeepSeek’s generation dynamics.

Notably, this corruption was absent from the zero-shot condition (0 occurrences) and the V1 five-shot condition (0 occurrences). It appeared only after the addition of `additional_info.md`, suggesting that the longer or more complex prompt may have destabilised DeepSeek’s output distribution in a way that occasionally produces out-of-vocabulary or cross-lingual token insertions.

4.2.4 FewShot-10-Random-V1

The 10-shot V1 condition doubled the number of exemplars to ten while keeping the same documentation configuration as 5-shot V1 (output expectations document only).

Table 4.5: FewShot-10-Random-V1 syntactic parsability

Model	Total	Parsable	Not Parsable	Parsability Rate
ChatGPT 5.0	90	90	0	100.0%
DeepSeek V3.1	90	87	3	96.7%
Gemini 2.5 Pro	90	90	0	100.0%

ChatGPT 5.0 and Gemini 2.5 Pro achieved perfect syntactic parsability, demonstrating that ten diverse exemplars without supplementary heuristics were already sufficient to teach them the full clingo grammar. DeepSeek V3.1 achieved 96.7% parsability, with 3 of 90 encodings failing.

Failure Analysis

The three DeepSeek failures fall into two categories, both consistent with patterns observed in earlier conditions.

Incorrect predicate names (2 failures). Two DeepSeek outputs use `st_informed(...)` instead of the correct `informed(...)`. The `st_` prefix is conventionally used for fluent and action predicates (`st_obs/3`, `st_hpd/3`), but `informed` is a static predicate that should not carry this prefix. The resulting encodings are:

```
st_informed("Nicole", "no bill needed", "the waitress"), true, 4).
st_informed("the waitress", "on the house", "Nicole"), true, 2).
```

The malformed structure—a parenthesised predicate list followed by `, true, 4)`—indicates that DeepSeek attempted to wrap `st_informed(...)` within the `st_hpd/3` or `st_obs/3` template, producing a hybrid that satisfies neither signature. This suggests a generalisation error in DeepSeek’s predicate-selection mechanism: it recognised that an event or state was being described but applied the wrong predicate template.

Unsafe variables (1 failure). One DeepSeek output contains the unsafe variable pattern observed in the five-shot condition:

```
member(X, "the book club members").
```

Here `X` appears only in this single fact with no body predicate to ground it, violating clingo’s safety requirement. This is the same failure mode documented in Section 4.2.2.

4.2.5 FewShot-10-Random-V2

The addition of `additional_info.md` to the 10-shot configuration—combining ten diverse exemplars with explicit encoding heuristics—achieved the best outcome across all conditions.

Table 4.6: FewShot-10-Random-V2 syntactic parsability

Model	Total	Parsable	Not Parsable	Parsability Rate
ChatGPT 5.0	90	90	0	100.0%
DeepSeek V3.1	90	90	0	100.0%
Gemini 2.5 Pro	90	90	0	100.0%

All three primary models produced outputs for all 90 target stories, and every encoding parsed successfully. This represents a 100% syntactic parsability rate across 270 generated encodings. Notably, the problems that plagued DeepSeek in earlier conditions—unsafe-variable errors in V1 five-shot and both unsafe-variable and character-corruption errors in V2 five-shot—were entirely eliminated. The inclusion of `additional_info.md` in the 10-shot configuration, combined with the larger exemplar set, was sufficient to close the remaining syntactic gap for DeepSeek while maintaining the perfect performance already achieved by ChatGPT and Gemini.

4.3 Discussion of Syntactic Results

The per-condition analyses presented in Sections 4.2.1 through 4.2.5 revealed distinct patterns of syntactic competence and failure across models and prompting configurations. In this section, we step back from the condition-by-condition narrative to address four cross-cutting

questions that synthesise the empirical findings: (1) which model achieved the highest overall syntactic validity, (2) how the number of few-shot exemplars affected syntactic correctness, (3) whether the supplementary encoding guide improved or harmed syntactic validity, and (4) what were the most common parser errors observed across all conditions.

4.3.1 Comparative Model Performance

To answer RQ2—how the three primary models compare on the syntactic validity dimension—we aggregate the parsability rates across all five API-based conditions. Table 4.7 presents the per-condition parsability rates for each model.

Table 4.7: Per-condition syntactic parsability rates for the three primary models. Weighted averages are calculated across all target stories for each model (ChatGPT: 450/470 parsable; DeepSeek: 430/470; Gemini: 450/470).

Condition	ChatGPT 5.0	DeepSeek V3.1	Gemini 2.5 Pro
ZeroShot-V1 (100 targets)	80.0%	99.0%	80.0%
FewShot-5-Random-V1 (95 targets)	100.0%	77.9%	100.0%
FewShot-5-Random-V2 (95 targets)	100.0%	84.2%	100.0%
FewShot-10-Random-V1 (90 targets)	100.0%	96.7%	100.0%
FewShot-10-Random-V2 (90 targets)	100.0%	100.0%	100.0%
Weighted Average	95.7%	91.5%	95.7%

ChatGPT 5.0 and Gemini 2.5 Pro achieved nearly identical weighted-average parsability rates of 95.7% and 95.7% respectively, with their only syntactic failures concentrated entirely in the zero-shot condition. DeepSeek V3.1 achieved a lower weighted average of 91.5%, despite being the only model to achieve near-perfect parsability in the zero-shot condition (99.0%). This result, with the best zero-shot performer recording the lowest overall average, illustrates the differential effect of prompting on each model’s syntactic output. ChatGPT and Gemini were brittle in the absence of exemplars but became perfect once exemplars were introduced. DeepSeek was near-perfect in isolation but became syntactically less reliable when exemplars and instructional documents were added to its context.

4.3.2 Effect of Shot Count on Syntactic Correctness

The relationship between shot count and syntactic correctness was neither monotonic nor uniform across models.

For ChatGPT and Gemini, the transition from zero-shot to any few-shot configuration was sufficient to eliminate all syntactic errors. Five exemplars were as effective as ten, and the addition of further exemplars neither improved nor degraded their performance. This suggests that these models did not require a large or diverse exemplar set to learn the syntactic conventions of the ASP output format; a minimal demonstration of the period-delimited convention, quoted-entity formatting, and predicate arity was sufficient.

For DeepSeek, the relationship was more complex and exhibited a U-shaped pattern. DeepSeek’s syntactic performance was strongest in the extreme conditions—zero-shot (99.0%) and the most richly specified condition, 10-shot V2 (100.0%)—and weakest in the intermediate conditions. In the 5-shot V1 condition, DeepSeek’s parsability dropped to 77.9%, and even with the supplementary encoding guide in the V2 configuration it rose only to 84.2%. The 10-shot configuration substantially recovered this deficit, achieving 96.7% in V1 and 100% in V2.

We interpret this pattern as follows. In the zero-shot setting, DeepSeek appears to operate from a default internal template that produces syntactically clean output. The introduction of five diverse exemplars disrupts this template without providing sufficient coverage to establish a new robust convention: DeepSeek learns that the examples use **X** as a variable in the context of rules with bodies, but fails to generalise the safety constraint that facts must not contain free variables. The result is the unsafe-variable errors documented in Section 4.2.2. By the time ten exemplars are provided, the broader coverage of syntactic patterns, including cases where unspecified entities are handled via default constants rather than variables, is sufficient to re-establish high parsability.

This nonlinear relationship underscores an important nuance for prompt engineering: *more* exemplars do not always produce *better* syntax, and models may pass through a region of degraded performance at intermediate shot counts before converging to a higher-quality output regime.

4.3.3 Effect of the Supplementary Encoding Guide

The `additional_info.md` document was introduced in the V2 variants of the 5-shot and 10-shot conditions to provide explicit heuristics for encoding edge cases, negation formatting, role compression, and unspecified-entity handling. Its effect on syntactic validity was not uniformly beneficial.

For ChatGPT and Gemini, `additional_info.md` had no measurable effect on syntactic parsability, as both models already achieved 100% in the V1 configurations. The document neither improved nor harmed their output.

For DeepSeek, the effect was condition-dependent and ambivalent. In the 5-shot setting, the encoding guide produced a modest improvement in overall parsability (from 77.9% to 84.2%), reducing the unsafe-variable error count from 21 to 2. However, this improvement was accompanied by the emergence of a novel and severe failure mode: the insertion of the Chinese character 极 into predicate tokens, string literals, and numeric arguments, accounting for 12 of the 15 remaining failures (Section 4.2.3). The net effect was an improvement of 6.3 percentage points in parsability, but the cost was the introduction of a qualitatively more severe error type as corrupted tokens are harder to diagnose and repair than unsafe variables.

In the 10-shot setting, the effect was unambiguously positive: DeepSeek’s parsability rose from 96.7% to 100%, and the character-corruption failure mode did not appear. The larger exemplar set appears to have stabilised DeepSeek’s output distribution sufficiently to prevent the cross-lingual token interference observed in the 5-shot V2 condition.

Table 4.8: Distribution of syntactic failure modes across models and conditions. A total of 79 invalid encodings were observed across 1,410 total predicted outputs (5.6% overall syntactic failure rate).

Failure Mode	Affected Models	Conditions	Occurrences
Missing terminating periods	ChatGPT, Gemini	ZeroShot-V1	40
Unsafe free variables	DeepSeek	FewShot-5-Random-V1, FewShot-5-Random-V2, FewShot-10-Random-V1	22
Chinese character corruption	DeepSeek	FewShot-5-Random-V2	12
Incorrect predicate names	DeepSeek	FewShot-10-Random-V1	2
Stray punctuation	DeepSeek	ZeroShot-V1	1
Missing output	DeepSeek	FewShot-5-Random-V1	2

This dual effect of `additional_info.md`, a partial syntactic improvement at the cost of a novel failure mode in one condition, and complete syntactic resolution in another, highlights a tension in supplementary documentation, at least for DeepSeek. The additional context provided useful syntactic guidance, as evidenced by the reduction in unsafe-variable errors from 21 to 2. However, the longer prompt also appeared to shift DeepSeek’s output distribution into less predictable regions, where token-level hallucinations became more likely. For ChatGPT and Gemini, neither effect was observed: both models maintained perfect parsability regardless of the encoding guide’s presence. The practical implication is that the effect of supplementary documentation is model-dependent: for some architectures it provides clear benefits when paired with adequate exemplar coverage, while for others it is simply unnecessary.

4.3.4 Common Parser Errors and Failure Mode Taxonomy

Synthesising the failure analyses from Sections 4.2.1 through 4.2.5, we identified six distinct categories of syntactic failure. Table 4.8 presents their distribution across models and conditions.

The failure modes fall into two broad classes. **Format-level errors**—missing periods and stray punctuation—account for 41 of 79 failures (51.9%) and are the only syntactic errors that affect ChatGPT and Gemini. These errors are purely surface-form issues: the predicates are internally well-formed (correct arity, balanced parentheses, properly quoted

strings) but fail to respect the delimiters required by the `clingo` parser. The uniformity of the missing-period pattern across 40 independent failures suggests that ChatGPT and Gemini share a common default output convention for ASP-like text: a space-delimited token sequence, that is incompatible with `clingo`’s grammar.

Content-level errors—unsafe variables, character corruption, incorrect predicate names, and missing outputs—account for 38 of 79 failures (48.1%) and are exclusive to DeepSeek. These errors reflect deeper issues in how the model maps natural language entities and actions to formal predicate structures. The unsafe-variable pattern (22 occurrences, the single most frequent DeepSeek failure) reveals a conceptual gap: the model understands that an entity exists (e.g., “food” was ordered) but fails to instantiate it with a concrete constant, substituting a free variable instead. The character-corruption pattern (12 occurrences) represents a qualitatively different failure mechanism—a token-level hallucination that produces syntactically malformed tokens rather than simply unsafe ones. Its appearance only in the condition where `additional_info.md` was added without adequate exemplar stabilisation (5-shot V2) marks it as a pathology of prompt-destabilised generation rather than a conceptual error.

The overall syntactic failure rate of 5.6% indicates that, across all conditions and models, approximately 1 in every 18 generated encodings is syntactically unparseable. However, this aggregate statistic masks the wide variation documented above: the failure rate ranges from 0% (in three of the five conditions for ChatGPT and Gemini, and in FewShot-10-Random-V2 for all models) to 22.1% (DeepSeek in FewShot-5-Random-V1). The sensitivity of these rates to prompting configuration, and the differential sensitivity across models, is a central finding of this syntactic evaluation.

In summary, the syntactic evaluation establishes that reliable ASP syntax generation is achievable across all three primary models under the right prompting configuration (ten diverse exemplars combined with explicit encoding heuristics). The performance ranking, weighted across conditions, places ChatGPT 5.0 and Gemini 2.5 Pro at 95.7% parsability, with DeepSeek V3.1 trailing at 91.5%. However, DeepSeek’s trajectory across conditions suggests that its syntactic competence is fundamentally high—it achieves 99% in zero-shot—but its output mechanism is more sensitive to prompt perturbation than its counterparts. The practical implication is that prompt engineering for DeepSeek requires particular attention to exemplar diversity and documentation stability: too few exemplars invite unsafe-variable errors, while supplementary documents in the absence of sufficient exemplars can trigger token-corruption pathologies. With adequate exemplar coverage and explicit heuristics, all three models achieve 100% syntactic validity, providing a reliable foundation for the semantic evaluation in Section 4.4.

4.4 Semantic Evaluation Results

While the syntactic evaluation in Section 4.2 established whether the generated encodings were parsable, it revealed nothing about whether they were *correct*—that is, whether the logical content of the encoding faithfully represented the narrative it was meant to capture.

To assess semantic accuracy, we conducted a manual annotation of sampled outputs using the rubric defined in Section 4.1.2 (Table 4.1). Each encoding was assigned a score from 0 to 5, with scores of 4 or higher considered passing.

For each experimental condition, we selected approximately 30 stories spanning the full range of similarity scores produced by the automated coarse-similarity filter described in Section 4.1.2. This sampling strategy ensured that the manual evaluation covered outputs of varying quality rather than focusing only on the best or worst cases. The resulting scores, pass rates, and qualitative patterns are reported below for each condition.

4.4.1 ZeroShot-V1

The zero-shot condition represents the most challenging configuration for the models: no exemplar translations were provided, and the only guidance came from the two supplementary instructional documents. Unsurprisingly, semantic accuracy was poor across all three models.

We manually annotated 10 same stories per model (30 annotations total), sampled from across the similarity range. Of these 30 annotations, only 4 received a passing score of 4. Even these passing encodings were far from ideal. They were considered acceptable only because they would produce valid answer sets in a downstream solver, not because they reflected the encoding conventions a domain expert would choose.

Table 4.9 presents the score distribution for each model.

Table 4.9: ZeroShot-V1 semantic score distribution and pass rates

Score	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
4 (pass)	2	0	2
3	6	6	1
2	2	4	7
Pass Rate	20.0%	0.0%	20.0%

ChatGPT 5.0 and DeepSeek V3.1 each achieved a 20% pass rate (2 of 10), while Gemini 2.5 Pro failed all 10 evaluated outputs. The majority of scores across all models fell in the 2–3 range, indicating encodings that captured some narrative elements correctly but contained two or more clear mistakes or hallucinated content.

While the score distributions differed modestly between models, the nature of the errors was remarkably consistent. We identified several systematic patterns that appeared across models and stories, contributing to the universally low pass rates.

A common pattern across all three models was the redundant encoding of entities using multiple parallel predicates. The most frequent manifestation was the simultaneous use of both `person/1` and `customer/1` to refer to the same individual. This redundancy, while not individually catastrophic, compounds across a full encoding, producing logic programs that are significantly more verbose than necessary. A domain expert would use only `customer/1`

for restaurant patrons, reserving **person/1** for contexts where the role is ambiguous. A related pattern was the use of multiple predicate forms for the same action. Several encodings contained both **pay/2** and **paid/1** to describe payment events. This behaviour of producing both the active and passive forms of the same logical relation suggests that the models are generating predicates based on surface-level lexical cues in the narrative text ("pay" appearing in the story) rather than selecting a single canonical representation for each event.

Another common behavior of all three models was their preference for the fluent **served/1** (via **st_obs**) over the action **put_down/3** (via **st_hpd**) when encoding a waiter delivering food to a table. While **served/1** is part of the target domain ontology, it captures only the customer's state, losing information about the agent, the food item, and the location that **put_down/3** encodes. Nearly every encoding that described food delivery used the fluent form, indicating that the models default to passive-state predicates rather than multi-argument action predicates.

Moreover, the models occasionally invented predicate names that have no counterpart in the established ontology. A recurrent pattern was the encoding of actions that were not explicitly described in the narrative. Models inferred **sit/1** when the character was already seated, added **prepare/3** when the story mentioned a chef but not the act of cooking, and in similar scenarios inferred **read_bill/1**, **enter/2**, and others. These inferences, while narratively plausible, violate the explicit-extraction principle: the models were instructed to encode only what the story states, not what can be reasonably assumed.

The overall pass rate of 13.3% (4 of 30) establishes a low baseline for semantic accuracy in the zero-shot condition. The multifaceted error patterns such as, redundant entity encoding, action-fluent confusion, hallucinated predicate names, and inferred actions indicate that the instructional documents alone were insufficient to teach the models the precise encoding conventions required for the ASP translation task.

4.4.2 FewShot-5-Random-V1

The introduction of five worked examples alongside the instructional documents produced a substantial improvement across all three models. Pass rates rose sharply from the zero-shot baseline, and the range of scores broadened to include the 4.5 category (better than ground truth) and occasional exact matches.

Table 4.10: FewShot-5-Random-V1 semantic score distribution and pass rates

Score	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
5 (exact match)	1	1	1
4.5 (better than GT)	2	8	5
4 (pass)	16	7	9
3	9	13	11
2	2	1	2
0	0	0	2
Pass Rate	63.3%	53.3%	50.0%

ChatGPT 5.0

ChatGPT 5.0 recorded the highest pass rate at 63.3% (19 of 30). Most of its passing encodings scored a 4, indicating correct but not flawless translations. The most common point of tension was the encoding of actions whose presence in the story was open to interpretation. In the ground truth, we sometimes encoded actions that were suggested but not explicitly stated—for instance, interpreting “visited” as an `enter` action. When the model did not encode such actions, the outcome depended on how clearly the action was conveyed. If the wording was genuinely ambiguous, the omission was considered acceptable and the encoding could still receive a passing score. If the action was reasonably clear and the model still omitted it, the encoding received a failing score. This gray area was the single most common reason for scores of 3 rather than 4. Other failures included using the `informed` predicate in scenarios that required `cancel_bill`, encoding the wrong person as the subject of an ordering action, and one incomplete encoding. On the lenient side, ChatGPT sometimes used the constant `r` for the restaurant and used `pick_up` in addition to or instead of `read_menu` for menu reading. Neither was penalized.

Gemini 2.5 Pro

Gemini 2.5 Pro achieved a 53.3% pass rate (16 of 30). While its pass rate was lower than ChatGPT’s, the quality of its passing encodings was notably higher: 8 of its 16 passing samples received a 4.5 score, meaning they were judged to be better than the ground truth. This was the highest rate of 4.5 scores across all three models in this condition.

One distinctive behavior noticed only with Gemini was its use of the constant `g` to group people in multi-character stories. The ground truth typically referred to such groups using a meaningful constant derived from the story narrative—for instance, `member("David", they)` and `member("his wife", they)` followed by `st_hpd(order(they, ...), true, ...)`. Gemini’s use of `g` instead was unconventional but functionally equivalent, and encodings that did this were still able to produce correct answer sets. It was not penalized.

Like ChatGPT, Gemini occasionally used `r` and `restaurant` interchangeably. Among the failing samples, the most common issue was missing actions—most frequently `put_down`,

followed by `lead_to` and `leave`. In two stories, Gemini assigned incorrect timesteps, including one where it gave the same timestep to a `put_down` and a `pay` action. One encoding received a score of 2 due to incorrect arity on the `order` predicate.

DeepSeek V3.1

DeepSeek V3.1 achieved a 50.0% pass rate (15 of 30), the lowest of the three. Like Gemini, it produced several encodings that outperformed the ground truth (5 scored 4.5). Its passing encodings shared the same flexibility as the other models—it used `r` and `restaurant` interchangeably, and used `w` for the waiter even when the story named the waiter explicitly.

Two observations were unique to DeepSeek. First, it tended to encode each food item in a shared order separately rather than grouping them under the constant `f` using `member/2`. This is not the optimal encoding but does not break the program. Second, it sometimes used the fluent `sitting` via `st_obs` instead of the action `sit` via `st_hpd`—again, functionally acceptable.

Among the failing samples, a common pattern was DeepSeek using group references like “we” or “they” in its predicates without first declaring them using `member/2`. In two exceptional stories involving complaints, it missed the `st_next` predicate entirely. In one cancellation scenario, it used `informed` instead of `cancel_bill`. Two encodings were missing entirely (scored 0)—one was absent from the response and one was null. Notably, a single encoding contained a Chinese character embedded in a string literal: `st_hpd(drink("Sarah", "[Chinese character] coffee"), true, 4)`.

Summary

Across all three models, the five-shot configuration marked a clear improvement over the zero-shot baseline. Pass rates of 50–63% demonstrate that even a small set of exemplars meaningfully improved semantic accuracy, and the appearance of 4.5 and 5 scores—absent entirely from the zero-shot condition—shows that the models were capable of producing high-quality translations. The most persistent challenges were the omission of non-explicit actions and occasional confusion between related predicates in the ontology.

4.4.3 FewShot-5-Random-V2

This condition replicated the five-shot random configuration but with the addition of `additional_info.md`, a supplementary document containing explicit heuristics for encoding edge cases, negation formatting, role compression, and unspecified-entity handling. The effect of this addition was not uniform across models. Gemini showed a dramatic improvement; ChatGPT showed no change; DeepSeek showed a slight decline, though not large enough to be conclusive.

Table 4.11: FewShot-5-Random-V2 semantic score distribution and pass rates

Score	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
5 (exact match)	2	1	0
4.5 (better than GT)	3	10	1
4 (pass)	13	16	12
3	11	3	16
2	1	0	1
Pass Rate	60.0%	90.0%	43.3%

ChatGPT 5.0

ChatGPT 5.0 achieved a 60.0% pass rate (18 of 30), essentially unchanged from the 63.3% it recorded in the V1 configuration. The addition of the supplementary encoding guide had no measurable effect on its output. The types of errors were also unchanged from the previous experiment. The most common issue was again the omission of actions when the story wording was not explicit—particularly **enter** and **leave**. Other recurring issues included timestep errors (such as assigning different timesteps to the **put_down** of two dishes that arrived together), not grouping related people using **member/2**, and one instance where the encoding included only one member of a group. One encoding inferred a **sit** action and another missed **st_next** in an exceptional story. On the acceptable side, ChatGPT sometimes declared food predicates after actions or fluents rather than before them, used **restaurant** when **r** would have sufficed, and omitted false actions when they were not explicit. None of these were penalized.

Gemini 2.5 Pro

Gemini 2.5 Pro achieved a 90.0% pass rate (27 of 30), a remarkable increase from 53.3% in V1. The inclusion of the supplementary document produced a roughly 37-point improvement, the largest single jump observed across any condition in this study. Of the 27 passing encodings, 10 received a 4.5 score, meaning they were judged better than the ground truth. The reasons for the 4.5 scores included: encoding an **enter** action that the ground truth had missed, making better choices between **sit** (action) and **sitting** (fluent) depending on the narrative context, more accurate timestep assignments, and encoding drinks and drink actions that the ground truth had omitted for convenience. In several cases, Gemini’s encodings of complex stories simply looked cleaner and more coherent than the ground truth, and in some instances Gemini helped us catch errors in the ground truth itself. Among the score 4 encodings, common issues were the same as in other experiments—using **r** and **restaurant** interchangeably. A novel behavior specific to this experiment was Gemini’s use of the string **c_group** to group customers, rather than a pronoun like “we” or “they” that would have been more aligned with the examples. This was unconventional but not penalized. Only 3 encodings failed, each for a distinct reason: one missed **st_next**, one

inferred a `sit` action, and one assigned different timesteps to eating actions that should have been simultaneous.

DeepSeek V3.1

DeepSeek V3.1 achieved a 43.3% pass rate (13 of 30), a decrease from 50.0% in V1. The decline is modest enough that it is not possible to attribute it definitively to the addition of the supplementary document. The error profile remained similar to the previous experiment. The most frequent issue in failing encodings was the inference of actions not present in the story, particularly `put_down` (the most common), followed by `eat`, `sit`, and `order`. DeepSeek also failed to declare `waiter/1` when the story explicitly mentioned a waiter but then used "waiter" in subsequent actions without the declaration, used `informed` where `cancel_bill` or `complain` was required, and missed `st_next` in exceptional stories. Among the passing encodings, common non-critical issues included not grouping people or food items where it made sense, and using both `pick_up` and `read_menu` when one would have sufficed. Chinese characters appeared in random positions in several encodings. These were obvious enough to be easily identified and removed, so they did not affect the semantic score, but they would have caused a syntax failure in an automated pipeline.

Summary

The divergent response to `additional_info.md` is the central finding of this condition. Gemini, which had been middle-ranked in V1, surged to the best performance observed across all conditions up to this point. ChatGPT’s output was essentially indifferent to the added instruction, and DeepSeek’s performance, if anything, slightly declined. This suggests differences in how each model integrates instructional text during generation: Gemini appears to prioritize explicit guidance over example patterns, while ChatGPT relies more heavily on the examples themselves. DeepSeek’s sensitivity to prompt changes is consistent with the pattern observed in the syntactic evaluation, where longer or more complex prompts could both help and destabilize its output.

4.4.4 FewShot-10-Random-V1

This condition doubled the exemplar count to ten while keeping the same documentation configuration as the 5-shot V1 condition. Only 29 of the 30 sampled stories were evaluated, as one (SID 15) had been included as an example in the input file.

Initial attempts with ChatGPT 5.0 and Gemini 2.5 Pro produced only 7 and 21 of the 90 target outputs respectively, due to output token limits. Supplementary trials were conducted with GPT-4o and Gemini Flash 2.5, which completed the full set but achieved pass rates of only 31% and 43%. After a later provider-side limit increase allowed the primary models to be re-run successfully (`max.tokens` raised from 6,000 to 32,000), their results improved substantially. The primary model results are reported below.

Table 4.12: FewShot-10-Random-V1 semantic score distribution and pass rates

Score	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
5 (exact match)	2	1	1
4.5 (better than GT)	7	9	6
4 (pass)	13	11	4
3	6	8	15
2	1	0	3
Pass Rate	~76%	~72%	~38%

ChatGPT 5.0 achieved its best performance across any condition so far at approximately 76% (22 of 29), a clear improvement from 63% in the 5-shot V1 configuration. Seven encodings received a 4.5 score, often for including actions or details that the ground truth had omitted. The error types were consistent with earlier experiments—inferring actions remained the most common reason for failing scores.

Gemini 2.5 Pro achieved approximately 72% (21 of 29), a meaningful improvement over its 5-shot V1 score of 53% but notably lower than the 90% it recorded in the 5-shot V2 condition where `additional_info.md` was provided. This reinforces a pattern observed across multiple conditions: for Gemini, high-quality instructions produce better results than simply increasing the number of examples. Its error profile was similar to ChatGPT’s in this condition, with inferred actions being the dominant failure mode.

DeepSeek V3.1 achieved approximately 38% (11 of 29), a decline from 50% in the 5-shot V1 configuration. This continues a pattern observed in both the syntactic and semantic evaluations: increasing the exemplar count can degrade DeepSeek’s output quality, particularly when the additional examples introduce encoding patterns that differ from the model’s default output template. The error types were unchanged from previous experiments.

4.4.5 FewShot-10-Random-V2

This condition combined the 10-example setup with `additional_info.md`, representing the richest prompting configuration tested. It produced the best or near-best semantic performance for all three models simultaneously, with an unusually high number of encodings judged to be better than the ground truth.

Table 4.13: FewShot-10-Random-V2 semantic score distribution and pass rates

Score	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
5 (exact match)	3	2	2
4.5 (better than GT)	9	9	5
4 (pass)	12	12	7
3	5	6	15
Pass Rate	82.8%	79.3%	48.3%

ChatGPT 5.0 achieved 82.8% (24 of 29), its best performance across all conditions. Three encodings were exact matches and nine received a 4.5 score, often for encoding details that the ground truth had omitted—drinks, the `in/2` fluent, read-menu actions, and better noun-string representations. The remaining 12 passing encodings scored 4, with only minor and now-familiar issues: using `r` instead of the full restaurant name, not grouping people or food using `member/2`, and occasionally using a less specific group reference. Only five encodings failed, each for narrow reasons—confusing `informed/3` with `complain/3`, missing `st_next`, inferring a `put_down` action, declaring two waiters in one story, and omitting one character’s `eat/2` action.

Gemini 2.5 Pro achieved 79.3% (23 of 29), strong performance that trailed only its own 90% in the 5-shot V2 condition. The distribution was nearly identical to ChatGPT’s: two exact matches, nine 4.5 scores for the same types of improvements (better noun representation, encoding drinks, adding `in/2`), and 12 scores of 4. The six failures were similarly narrow in scope—assigning the same timestep to `enter` and `sit`, giving different timesteps for simultaneous eating, using `f` inappropriately in an order action, and inferring a `sitting` fluent. The marginal drop from 5-shot V2 to 10-shot V2 reinforces a pattern observed throughout this study: Gemini achieves peak performance with high-quality documentation and a moderate number of examples; doubling the example count adds little beyond what the instructional document already provides.

DeepSeek V3.1 achieved 48.3% (14 of 29), an improvement from the 10-shot V1 configuration (38%) but still below its 5-shot V1 result of 50%. Two encodings were exact matches and five received 4.5 scores. The 15 failures reflected persistent challenges: inferring actions (`put_down`, `eat`, `order`, `drink`) and inventing predicates outside the target ontology (`join/2` appeared across multiple stories, along with `standing_by/1` and `claim_reservation/1`), as well as the familiar `st_next` omission and `informed-for-complain` substitution. The improvement from the V1 10-shot configuration suggests that `additional_info.md` offers some stabilizing effect when paired with a larger exemplar set, but the gain was not sufficient to surpass its simpler 5-shot V1 result. DeepSeek’s sensitivity to prompt configuration—where more is not always better—remains a limiting factor.

Taken together, the 10-shot V2 condition produced the best overall outcomes across the three models considered jointly. ChatGPT and Gemini both exceeded 79% pass rates with many outputs outperforming the ground truth, and DeepSeek improved from its 10-shot V1 result, narrowing roughly a third of the gap between its V1 performance and the leading

two models. This configuration—ten diverse exemplars combined with explicit encoding heuristics—represents the most reliable prompting strategy for achieving both syntactic and semantic quality in the ASP translation task.

4.4.6 Manual-Prompting (Web Interface)

As an exploratory supplement to the API-based experiments, we conducted a trial using the official web chat interfaces of each model vendor. The model versions available through these interfaces at the time were GPT-5.1, Gemini 3.0 Pro, and DeepSeek V3.2, all newer releases than the versions used in the API experiments. The prompt was pasted directly into the chat and requested all 100 stories in a single response, with the output format simplified to only `sid` and `logic_form` fields. There was no automated batching, no JSON repair layer, and no programmatic retry logic.

ChatGPT 5.1 declined to generate all 100 outputs in one response, stating that doing so would exceed output limits. After several clarifying exchanges, the task was divided into three conversational chunks: SIDs 0–33, 34–66, and 67–99. Gemini 3.0 Pro and DeepSeek V3.2 produced their full outputs in a single response without objection.

Table 4.14: Manual-prompting (web interface) semantic score distribution and pass rates.

Score	ChatGPT 5.1	Gemini 3.0 Pro	DeepSeek V3.2
5 (exact match)	1	3	1
4.5 (better than GT)	1	6	2
4 (pass)	7	15	15
3	6	5	11
2	3	1	1
1	11	0	0
0	1	0	0
Pass Rate	30.0%	80.0%	60.0%

ChatGPT 5.1 achieved only 30.0% (9 of 30), but the aggregate obscures a sharp bifurcation within its output. In the first two conversational chunks (SIDs 0–66), ChatGPT performed adequately, with 8 encodings receiving passing scores and errors resembling those observed in earlier experiments. In the third chunk (SIDs 67–99), however, ChatGPT departed entirely from the input data and generated encodings for entirely fabricated stories that bore no relation to the target narratives. Eleven of these encodings received a score of 1, indicating entirely fabricated content unrelated to the input narratives. One was missing entirely. This catastrophic failure mode, full hallucination of new stories in place of input narratives, was unique to this experiment and to this model. No other model in any condition substituted invented stories for the provided text. A plausible contributing factor is the cumulative conversational context of the chat interface: unlike the stateless API experiments, where each batch received the full prompt and examples fresh, the web-based

interaction relied on the model maintaining coherence across multiple conversational turns, which appears to have degraded generation quality in the final chunk.

Gemini 3.0 Pro achieved 80.0% (24 of 30), the strongest performance in this condition. Three encodings were exact matches and six received a 4.5 score for improvements over the ground truth distributed across a variety of stories. Only five encodings failed, each for narrow reasons, such as missing `st_next`, a false order action, and misusing group references. No hallucinations were observed, and the output was clean and consistent throughout all 100 stories despite being produced in a single response. This continues the pattern observed across experiments: Gemini’s instruction-following mechanism appears robust to changes in delivery modality, model version, and prompt format.

DeepSeek V3.2 achieved 60.0% (18 of 30), intermediate between the other two models. The error profile was familiar from the API experiments: timestep errors, using group references without declaring them via `member/2`, inferring actions (particularly `put_down` and `sit`), redundant encoding of both `sit` and `sitting` for the same action, and missing `st_next` in exceptional stories. No hallucinations were observed. DeepSeek was the most consistent model across delivery mechanisms; its error types and overall pass rate in the web-based trial closely resembled its API results.

These results reinforce a pattern observed across the API experiments: Gemini is the most robust to changes in delivery mechanism, DeepSeek is consistent but error-prone at a moderate level, and ChatGPT is sensitive to context management—performing well under stateless conditions but susceptible to degradation when conversational history accumulates. For deployment scenarios requiring reliable structured output from LLMs, stateless prompting with context re-injection appears preferable to conversational interfaces.

4.4.7 Summary of Semantic Results

The six experimental conditions described above form a matrix across two dimensions: exemplar count (0, 5, or 10) and the presence or absence of the supplementary encoding guide (`additional_info.md`). Table 4.15 consolidates the pass rates for all three models across all configurations.

Table 4.15: Consolidated semantic pass rates across all conditions

Condition	ChatGPT 5.0	Gemini 2.5 Pro	DeepSeek V3.1
ZeroShot-V1	20.0%	0.0%	20.0%
FewShot-5-Random-V1	63.3%	53.3%	50.0%
FewShot-5-Random-V2	60.0%	90.0%	43.3%
FewShot-10-Random-V1	75.9%	72.4%	37.9%
FewShot-10-Random-V2	82.8%	79.3%	48.3%
Manual-Prompting (web)	30.0%	80.0%	60.0%

Note: Bold values indicate the best performance for each model.

ChatGPT 5.0

ChatGPT 5.0 improved steadily as the prompt configuration became more informative. Its pass rate rose from 20% in the zero-shot condition to 63% with five examples, then to 76% and 83% in the 10-shot configurations. The supplementary encoding guide had minimal effect on its output—the V1-to-V2 changes were small in both the 5-shot (63% to 60%) and 10-shot (76% to 83%) settings, suggesting that ChatGPT learns predominantly from the examples rather than from explicit instructional text. Its best API result (83% in 10-shot V2) was the second-highest pass rate across all API-based conditions, behind Gemini’s 90% in the 5-shot V2 configuration. The web-based trial was the notable exception: ChatGPT’s performance collapsed to 30% due to a catastrophic hallucination of entirely fabricated stories in the final conversational chunk, a failure mode not observed in any other experiment.

Gemini 2.5 Pro

Gemini 2.5 Pro showed the most dramatic improvement from the zero-shot baseline (0%) to its peak performance (90% in 5-shot V2). Critically, Gemini’s best result was not achieved with the most examples, but with the combination of moderate exemplars (5) and the supplementary encoding guide. When the guide was absent (5-shot V1), its pass rate was only 53%; adding the guide (5-shot V2) produced a 37-point jump to 90%. In the 10-shot configurations, the guide still helped, but the improvement was smaller—from 72% to 79%—suggesting that a larger exemplar set partially substitutes for explicit instructions. Across both API and web-based delivery, Gemini was the most consistent model, achieving 80% in the web trial without any of the hallucination issues that affected ChatGPT.

DeepSeek V3.1

DeepSeek V3.1 had the most variable trajectory. Its zero-shot pass rate (20%) tied ChatGPT, but subsequent performance depended unpredictably on the prompt configuration. It peaked at 50% in the simplest few-shot condition (5-shot V1) and declined as the prompt grew more complex—43% in 5-shot V2, 38% in 10-shot V1, then a partial recovery to 48% in 10-shot V2. DeepSeek was the only model for whom increasing the number of examples or adding instructional text could degrade output quality. It also struggled with predicate invention: across multiple conditions, DeepSeek generated predicate names such as `join`, `invite`, `bring`, `choose`, `standing_by`, `claim_reservation`, and `charged` that do not appear in the target ontology. The web-based trial (60%) was DeepSeek’s best relative showing, narrowing the gap behind Gemini.

Persistent Error Patterns

Across all experiments, the most persistent error patterns were the omission of actions that were not narratively explicit (particularly `enter`), the invention of predicates outside the established domain ontology, confusion between related predicates (such as `informed` versus `complain` or `cancel_bill`), incorrect timestep assignment, the failure to declare group

memberships via `member/2` before using group references, and missing `st_next` in exceptional complaint scenarios.

Chapter 5

Conclusion and Future Work

This thesis set out to answer a simple question: Can Large Language Models reliably translate unstructured natural language narratives into the structured, logic-compatible representations required by formal reasoning systems? The question is deceptively simple because it sits at the intersection of two research traditions that have historically operated under very different assumptions: statistical language modeling, which thrives on the ambiguity and variability of natural language, and symbolic logic programming, which demands precision, consistency, and strict adherence to syntactic form.

The experimental work presented in the preceding chapters demonstrates that the answer to this question is a qualified yes. Under the right prompting conditions, state-of-the-art LLMs can achieve both high syntactic validity and strong semantic accuracy in the narrative-to-ASP translation task. However, the qualification matters: the “right” conditions are model-specific, and the path to reliable performance is neither linear nor uniform across architectures.

5.1 Summary of Findings

The study evaluated three models—GPT-5, Gemini 2.5 Pro, and DeepSeek V3.1—across five prompting configurations that varied the number of worked examples and the presence of a supplementary encoding guide. The evaluation assessed both syntactic validity (whether the generated ASP encodings could be parsed by the `clingo` grounder) and semantic accuracy (whether the encodings faithfully captured the narrative content as judged by human annotators).

Syntactic Validity. The syntactic evaluation, conducted exhaustively across all 1,410 generated encodings, revealed that both ChatGPT and Gemini achieved weighted-average parsability rates exceeding 95%, with their only failures concentrated in the zero-shot condition where they defaulted to a space-delimited rather than period-terminated output format. DeepSeek, despite recording the single best performance of any model (99% in the zero-shot condition), proved more sensitive to prompt configuration: its parsability dropped to 76.6% in the 5-shot V1 condition due to unsafe free variables, and a novel character-corruption failure mode emerged when the supplementary encoding guide was introduced in the 5-shot V2 condition. The richest prompting configuration—10-shot with the supplementary guide—eliminated all syntactic errors across all three models, achieving 100% parsability for 270 encodings.

Semantic Accuracy. The semantic evaluation, conducted via manual annotation of

30 sampled encodings per model per condition using a 0–5 rubric, revealed substantial variation in how different models responded to prompting interventions. ChatGPT improved steadily as prompt richness increased, from 20% in the zero-shot condition to 83% in the 10-shot V2 configuration. Gemini showed the most dramatic single improvement—from 53% to 90%—when the supplementary encoding guide was added to the 5-shot configuration, surpassing its own 10-shot performance. DeepSeek achieved its best result (50%) in the simplest few-shot condition and exhibited a U-shaped performance curve, with output quality initially degrading as prompts grew more complex before partially recovering in the richest configuration. The web-based trials, conducted with newer model versions, confirmed Gemini’s consistency across delivery mechanisms while revealing that ChatGPT’s conversational interface introduced a hallucination failure mode not observed in the API-based experiments.

5.2 Addressing the Research Questions

The four research questions posed in Chapter 3 can now be answered in light of the empirical findings.

RQ1: How can we design a structured pipeline that transforms narrative text into logic-compatible representations suitable for ASP? The experimental framework developed in this thesis—comprising a standardized input JSON format, batched API inference with global context injection, automated `clingo`-based syntactic validation, and a stratified manual semantic annotation protocol—represents a functioning and reproducible pipeline. The batched inference strategy successfully overcame output token limitations, and the integration of a JSON repair layer mitigated the nested-quoting problems that arose from embedding ASP predicates within JSON fields. The key design insight is that the pipeline must accommodate the stateless nature of API-based LLM inference: few-shot examples must be re-injected into every batch, and structural formatting rules must be explicit and repeated.

RQ2: How do different LLMs compare in their ability to produce syntactically valid and semantically accurate ASP encodings? The comparison revealed that no single model dominated across all conditions. ChatGPT was the most consistent performer, improving monotonically with richer prompts and achieving the highest API-based semantic pass rate (83%). Gemini was the most responsive to instructional documentation, producing the single best result (90%) when the supplementary encoding guide was paired with a moderate exemplar count. DeepSeek was the most variable, achieving strong zero-shot performance but struggling with prompt configurations that improved the other models. These differences suggest that model selection and prompt design should be treated as a joint optimization problem rather than as independent choices.

RQ3: What prompting strategies yield the most ASP-compatible narrative outputs? The richest configuration—10 diverse exemplars combined with the supplementary encoding guide (`additional_info.md`)—produced the best overall outcomes across models, with ChatGPT and Gemini both exceeding 79% semantic pass rates and all three models achieving 100% syntactic validity. However, the path to this result differed by model.

For ChatGPT, more exemplars consistently helped and the supplementary guide had minimal effect. For Gemini, the supplementary guide was transformative, producing a 37-point improvement in the 5-shot condition, which was a larger gain than doubling the exemplar count. For DeepSeek, more was not always better: its best result came from the simplest few-shot configuration. The practical implication is that prompt engineering for this task should be model-aware rather than one-size-fits-all.

RQ4: What are the most common structural and logical deficiencies in LLM-generated outputs? The syntactic and semantic evaluations revealed six distinct categories of error. Syntactic failures fell into format-level errors (missing terminating periods, stray punctuation) and content-level errors (unsafe free variables, character corruption, incorrect predicate names, missing outputs). Semantic failures, which grew less frequent but did not fully disappear in richer prompting configurations, were dominated by four patterns: the omission of actions not explicitly stated in the narrative (particularly *enter* and *leave*), the invention of predicates outside the target ontology, confusion between semantically related predicates (e.g., *informed* versus *complain* or *cancel_bill*), and incorrect timestep assignment. These persistent errors highlight the fundamental tension between the extraction-oriented task design and the models’ tendency toward commonsense inference.

5.3 Broader Implications

Beyond the specific findings, this work carries several implications for the broader integration of neural and symbolic approaches to AI.

First, the results demonstrate that the knowledge acquisition bottleneck—a longstanding challenge for logic-based reasoning systems—can be meaningfully addressed using LLMs as structured information extractors. The RestKB library and similar domain-specific knowledge bases have traditionally required labor-intensive manual encoding. The 83–90% semantic pass rates achieved by ChatGPT and Gemini under optimal prompting conditions suggest that a human-in-the-loop workflow, where LLMs generate initial encodings that are subsequently verified and corrected by domain experts, could substantially accelerate the development of structured knowledge bases for formal reasoning.

Second, the differential response of models to prompting interventions underscores the importance of systematic comparative evaluation. A researcher or practitioner who tested only one model or one prompting configuration might draw very different conclusions than one who surveyed the full matrix. The finding that DeepSeek’s performance degraded when the supplementary encoding guide was introduced—while Gemini’s improved by 37 points—is a cautionary example. Prompt engineering is not a monotonic lever; additional context can shift model outputs into less predictable regions of the generation distribution.

Third, the emergence of the 4.5 score category—encodings judged better than the ground truth—was an unexpected but encouraging finding. It occurred with surprising frequency, particularly from Gemini in the conditions where it performed best. This suggests that LLMs are not merely mimicking the encoding patterns shown in examples but are capable of making reasonable interpretive choices that, in some cases, improve upon the consistency or

completeness of human-annotated ground truth. This finding supports a vision of human-AI collaboration where the model’s output serves not as a final product but as a proposal that can both save human effort and surface potential improvements to existing annotations.

5.4 Limitations

The contributions of this thesis should be considered alongside several limitations that define the boundaries of its claims.

Domain Specificity. The study was conducted entirely within the restaurant dining domain, using a target ontology derived from the RestKB framework. While this domain provides a useful testbed—it is sufficiently rich to admit both routine and exceptional narratives—the generalizability of the findings to other domains (medical narratives, legal case descriptions, technical procedures) remains unverified. The predicate inventory, the types of actions and fluents, and the nature of implicit information all vary substantially across domains, and model performance may differ accordingly.

Language and Cultural Scope. All narratives were in English and reflected predominantly Western dining customs. The models’ ability to handle narratives in other languages, or narratives rooted in cultural contexts with different dining norms, was not assessed. Given the known cultural biases in LLM training data, this is a non-trivial limitation for applications that aspire to cross-cultural robustness.

Model Versioning. The primary experiments were conducted with specific model versions available via the OpenRouter API in late 2025. LLMs evolve rapidly, and the specific failure modes documented here, such as the missing periods, the Chinese character corruption, the unsafe variables, may be addressed in subsequent releases. The web-based trials, conducted with newer model versions (GPT-5.1, Gemini 3.0 Pro, DeepSeek V3.2), showed that ChatGPT’s performance had dramatically changed between versions, highlighting both the value and the perishability of benchmark data in this fast-moving field.

Evaluation Methodology. The semantic evaluation, while carefully designed, was inherently subjective and labor-intensive. The 0–5 rubric, with its distinction between scores of 3 and 4, necessarily involved judgment calls about what constitutes a “major” versus “minor” error. The stratified sampling strategy, while more representative than simple random sampling, still covered only 30 of approximately 90–100 outputs per model per condition. A larger annotation effort could yield more precise pass-rate estimates.

Pipeline Integration. The evaluation stopped at the point of assessing the quality of the generated ASP encodings. The encodings were not fed into a downstream reasoning system to verify that they actually enabled correct inference on tasks such as question answering, consistency checking, or intention recognition. Syntactic validity and semantic accuracy are necessary conditions for successful downstream reasoning, but they are not sufficient.

5.5 Future Work

The findings and limitations of this thesis open several promising directions for future research.

Cross-Domain Generalization. The most immediate extension of this work is to replicate the experimental framework in one or more additional narrative domains. Medical visit narratives, legal case summaries, or technical troubleshooting stories would test whether the prompting strategies identified as effective in the restaurant domain transfer to contexts with different predicate vocabularies, event structures, and types of implicit information. A cross-domain study would also reveal whether model rankings are domain-specific or stable across narrative types.

End-to-End Reasoning Evaluation. A natural next step is to close the loop by feeding LLM-generated ASP encodings into a downstream solver and measuring the correctness of the resulting inferences. This would address the sufficiency question that this thesis left open: are syntactically valid and semantically accurate encodings sufficient to enable correct reasoning? It would also surface a new class of errors—encodings that parse correctly and look plausible to a human annotator but produce incorrect or incomplete answer sets when queried.

Fine-Tuning. The experiments in this thesis relied exclusively on in-context learning via prompting. Fine-tuning LLMs on a dataset of narrative-to-ASP translation pairs—potentially using the 100-story dataset developed here as training data—could yield substantial improvements, particularly for DeepSeek, which showed the most variability under different prompting conditions. Comparing the cost-effectiveness of fine-tuning versus sophisticated prompt engineering would provide practical guidance for practitioners.

Automated Semantic Evaluation. The manual annotation protocol, while necessary given the underspecification of natural language with respect to logical form, was a bottleneck. Developing automated or semi-automated semantic evaluation methods, perhaps by comparing the answer sets produced by predicted and ground-truth encodings rather than comparing the encodings directly, could enable evaluation at much larger scale and with greater consistency.

Multi-Modal Narrative Understanding. The narratives in this study were purely textual. Many real-world narratives involve multiple modalities—spoken dialogue with visual context, written reports with accompanying diagrams, or instructional videos with narration. Extending the pipeline to handle multi-modal inputs, where an LLM processes both text and images to produce a unified ASP encoding, would move closer to the demands of practical applications such as robotic instruction following or interactive storytelling systems.

5.6 Closing Remarks

This thesis began with the observation that narratives are central to human communication—rich with events, intentions, and causal relationships that are intuitive for us but challenging for machines. The complementary strengths of Large Language Models and logic

programming systems offer a promising path toward computational narrative understanding that is both linguistically flexible and logically rigorous.

The experimental work presented here demonstrates that this path is viable. Under carefully designed prompting conditions, state-of-the-art LLMs can serve as effective translators between the unstructured world of natural language stories and the structured world of formal logic. The translation is not perfect—models still omit actions, invent predicates, and confuse related concepts—but it is good enough to suggest a future in which the knowledge acquisition bottleneck for logic-based reasoning systems is substantially alleviated.

Perhaps the most encouraging finding is not any single performance number but the pattern of 4.5 scores—the encodings judged to be better than the ground truth. These moments, where a model’s interpretive choice improved upon the human annotation, hint at a collaborative relationship between human knowledge engineers and AI systems. The goal is not to replace human expertise but to amplify it: to let models handle the routine translation work while humans focus on edge cases, domain modeling, and the design of reasoning rules that give structured knowledge its power.

The integration of neural and symbolic approaches to AI is still in its early stages, and much work remains to be done. But the results of this thesis suggest that the bridge between these two traditions is not only conceptually appealing but practically constructible, with one carefully engineered prompt at a time.

Acknowledgments This document was copy-edited with assistance from the following AI tools: [AI Tool Name] The content has been reviewed and edited by a human. The conceptual flowchart in Figure X was generated using [AI Tool Name], version X.X, and subsequently edited for accuracy and contextual alignment by the researcher.

Appendix A

Encoding Schema Specification

Full Encoding Schema

This appendix provides the complete encoding schema used for annotating all ground-truth logic forms and for instructing the Large Language Models during the experiments. The schema defines the predicate inventory, entity and fluent types, action vocabulary, naming conventions, and special-case predicates that govern the translation of natural language restaurant narratives into Answer Set Programming (ASP) facts.

General Conventions

Any strings from the original story representing names of people, restaurants, food items, etc. are placed within double quotes in the ASP encoding (e.g., "Allie", "the waiter", "miso soup").

Entity Predicates

The following unary predicates are used to declare static entities mentioned in the narrative:

Predicate	Description
restaurant/1	A restaurant entity
customer/1	A customer
person/1	A person (generic role, not a customer)
food/1	A food dish
waiter/1	A waiter or waitress
beverage/1	A beverage or drink
host/1	A host or hostess
cook/1	A cook or chef

Group Membership. When customers form a group referred to collectively as “they” or a similar pronoun, and they perform an action together (e.g., entering the restaurant), or when a collection of food/drinks is ordered at the same time without specifying which individual will consume each item (e.g., a dish to share for the whole table), the binary

predicate `member/2` is used. The fact `member(X, Y)` means that item `X` (a person or dish) is part of group `Y`.

Temporal and Action Predicates

The temporal backbone of the encoding uses two core predicates:

- `st_hpd(Action, true, I)` — The specified `Action` occurred at time step `I`, where `I` ranges from 0 to n inclusive. The second argument is always the constant `true` for positive occurrences.
- `st_obs(Fluent, Boolean, I)` — The specified `Fluent` holds (or does not hold) at time step `I`. The second argument is either `true` or `false`.

Negative Actions. When an action is explicitly stated as *not* occurring at any point in the story, it is encoded as:

`st_hpd(Action, false).`

Consecutive Time Steps. The predicate `st_next(I1, I2)` asserts that story steps `I1` and `I2` represent consecutive moments in real time. This predicate is used in exceptional scenarios, particularly when a customer complains. If a complaint occurs at story step `I`, a fact `st_next(I-1, I)` must be included.

Reserved Constants

The following constants are used to represent unnamed or generic entities in the narrative:

Constant	Meaning
<code>t</code>	The table
<code>b</code>	The bill
<code>r</code>	The restaurant (if unnamed in the story)
<code>w</code>	The waiter (if unnamed in the story)
<code>f</code>	An order consisting of several dishes
<code>tip</code>	The tip or gratuity

Fluents (Observable States)

The following fluents may be specified using the `st_obs/3` predicate. Each fluent captures a state that holds at a given time step.

Fluent	Description
<code>in(C, R)</code>	Customer <code>C</code> is in restaurant <code>R</code>
<code>welcomed(C)</code>	Customer <code>C</code> has been welcomed by the host or waiter
<code>at_l(X, L)</code>	Thing or person <code>X</code> is at location <code>L</code>
<code>sitting(P)</code>	Person <code>P</code> is sitting
<code>standing_by(P, L)</code>	Person <code>P</code> is standing by location <code>L</code> (e.g., next to the table)
<code>holding(P, T)</code>	Person <code>P</code> is holding thing <code>T</code>
<code>hungry(C)</code>	Customer <code>C</code> is hungry
<code>paid(B)</code>	Bill <code>B</code> has been paid
<code>open(R)</code>	Restaurant <code>R</code> is open
<code>informed(P1, S, P2)</code>	Person <code>P1</code> has transmitted information <code>S</code> to person <code>P2</code>
<code>bill_generated(C, B)</code>	Bill <code>B</code> has been generated for customer <code>C</code>
<code>food_prepared(Ck, F, W)</code>	Cook <code>Ck</code> has prepared food <code>F</code> for waiter <code>W</code>
<code>available(F, R)</code>	Food/dish <code>F</code> is available at restaurant <code>R</code>
<code>served(C)</code>	Customer <code>C</code> has been served
<code>menu_read(C)</code>	Customer <code>C</code> has read the menu

Actions (Specifiable via `st_hpd`)

The following actions may appear as the first argument of the `st_hpd/3` predicate. Unless otherwise noted, all actions are drawn from explicit statements in the narrative.

Action	Description
<code>enter(P, R)</code>	Person P enters restaurant R
<code>greet(P, C)</code>	Person P (host or waiter) greets customer C
<code>lead_to(P, C, L)</code>	Person P leads customer C to location L
<code>move(P, L1, L2)</code>	Person P moves from location L1 to L2
<code>sit(P)</code>	Person P sits down
<code>pick_up(P, T, L)</code>	Person P picks up thing T from location L
<code>read_menu(P)</code>	Person P reads the menu
<code>put_down(P, T, L)</code>	Person P puts down thing T at location L
<code>order(P, X, W)</code>	Person P orders food/drink X from waiter W
<code>order_for(P1, P2, X, W)</code>	Person P1 orders food/drink X for person P2 via waiter W
<code>prepare(Ck, F, W)</code>	Cook Ck prepares food/dish F requested by waiter W
<code>eat(P, F)</code>	Person P eats food/dish F
<code>drink(P, D)</code>	Person P drinks beverage D
<code>request(P1, T, P2)</code>	Person P1 requests thing T (object or action) from person P2
<code>read_bill(P)</code>	Person P reads the bill/check
<code>pay(P, B)</code>	Person P pays bill B
<code>pay(P, T)</code>	Person P pays tip T (where T is the constant <code>tip</code>)
<code>stand_up(P)</code>	Person P stands up
<code>leave(P)</code>	Person P leaves the restaurant
<code>order(P)</code>	Person P orders (used in negative form: <code>st_hpd(order(P), false)</code>)
<code>complain(C, W)</code>	Customer C complains to waiter W
<code>cancel_bill(P)</code>	Person P (e.g., waiter or owner) cancels the bill

Actions Not Explicitly Mentioned in Narratives. The following actions are part of the schema but are not expected to appear verbatim in a story. They represent external events that may need to be inferred or supplied for downstream reasoning:

Action	Description
<code>make_unavailable(F, R)</code>	Food/dish F becomes unavailable at restaurant R (due to an external event)
<code>interference</code>	An interference in the communication between two actors has occurred

Appendix B

Supplementary Encoding Guide

This appendix reproduces the supplementary encoding guide (`additional_info.md`) that was provided to the models in the V2 experimental conditions (FewShot-5-Random-V2 and FewShot-10-Random-V2). This document supplemented the output expectations schema (Appendix A) with practical heuristics for handling edge cases, encoding ambiguities, and common pitfalls in the translation of natural language narratives into ASP logic forms.

B.1 Core Principle

Use **only** the predicates provided in the output expectations document and in the examples provided. Do **not** assume a predicate exists if it is not explicitly listed.

B.2 Order Specifications

Separate Food and Drink: Drinks and food in a restaurant order must be encoded separately. They **cannot** be combined as a single food item or order.

B.3 Inference Rules

No Unmentioned Actions: If a story does **not** explicitly mention actions (e.g., entering a restaurant, ordering, eating), do **not** infer them. Leave them out of the encoding. Inference is handled by the pre-existing ASP code.

Negated Activities: For negations in a story (e.g., “They did not eat the salad,” “Sam forgot to pay”), use:

- `st_obs(Fluent, false, I)`
- `st_hpd(Action, false)`

For false actions specifically: Do **not** include a timestep. Use only `st_hpd(Action, false)`.

B.4 Entity Representation

Waiter Role: If someone other than an explicitly named waiter (e.g., manager, owner, host) performs waiter duties (greeting, serving, etc.), encode them as `waiter("the manager")`, `waiter("owner")`, etc.

Exception: If the story *also* mentions a separate “waiter,” then all other such people must be represented by the `person()` predicate instead.

Unspecified Food Items:

- If the story mentions a generic word like “food” or “meal” but no specific item name, encode as `food("food")`, `food("meal")`, etc.
- If even generic words are absent, do **not** encode `food("food")`. Use the constant `'f'` wherever necessary in the encodings.

B.5 General Guidelines

Inconsistencies: If you find inconsistencies in the given examples, follow the encoding method that appears **most frequently**.

Unnecessary Predicate: The `story_step/1` predicate is **not** required and can be safely omitted. Its presence or absence does not affect correctness.

References

- [1] A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- [2] Tom B Brown. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.
- [3] Yuheng Wang. *Knowledge Authoring with Factual English, Rules, and Actions*. PhD thesis, Stony Brook University, 2023.
- [4] Sarah E. Worth. Storytelling and narrative knowing: An examination of the epistemic benefits of well-told stories. *The Journal of Aesthetic Education*, 42:42 – 56, 2008.
- [5] Qinglin Zhang, Chris Benton, and Daniela Inclezan. An application of ASP theories of intentions to understanding restaurant scenarios: Insights and narrative corpus. *Theory and Practice of Logic Programming*, 20(2):273–293, April 2019.
- [6] Roger C. Schank and Robert P. Abelson. Scripts, plans, goals and understanding: an inquiry into human knowledge structures. 1978.
- [7] Erik T. Mueller. Understanding script-based stories using commonsense reasoning. *Cogn. Syst. Res.*, 5(4):307–340, 2004.
- [8] Erik T. Mueller. Modelling space and time in narratives about restaurants. *Lit. Linguistic Comput.*, 22(1):67–84, 2007.
- [9] Chitta Baral and Michael Gelfond. Reasoning about intended actions. In *AAAI Conference on Artificial Intelligence*, 2005.
- [10] Daniela Inclezan, Qinglin Zhang, Marcello Balduccini, and Ankush Israney. Understanding restaurant stories using an ASP theory of intentions. In *International Conference on Logic Programming*, 2017.
- [11] Hamaneh Hajishirzi and Erik T. Mueller. Question answering in natural language narratives using symbolic probabilistic reasoning. In *The Florida AI Research Society*, 2012.
- [12] F. van Harmelen, V. Lifschitz, and B. Porter. *Handbook of Knowledge Representation*. Elsevier, 2008.

- [13] Walter Kintsch. The representation of knowledge and the use of knowledge in discourse comprehension. 1989.
- [14] C. Grosan and A. Abraham. Knowledge representation and reasoning. In *Intelligent Systems: A Modern Approach*, pages 131–147. 2011.
- [15] Lavanya Sharma and Pradeep K. Garg. Knowledge representation in artificial intelligence: An overview. *Artificial Intelligence*, 2021.
- [16] John Mylopoulos and Hector J. Levesque. An overview of knowledge representation. In *On Conceptual Modelling*, 1983.
- [17] Bob Carpenter and Richmond H. Thomason. Inheritance theory and path-based reasoning: An introduction. 1990.
- [18] Robert F. Simmons and Jonathan Slocum. Generating English discourse from semantic networks. *Commun. ACM*, 15:891–905, 1972.
- [19] Daphne Koller and Avi Pfeffer. Probabilistic frame-based systems. In *AAAI/IAAI*, 1998.
- [20] Ye-Yi Wang, Li Deng, and Alex Acero. Semantic frame-based spoken language understanding. 2011.
- [21] D. Metaxas and T. Sellis. A database implementation for large frame-based systems. *Proceedings. Second International Conference on Data and Knowledge Systems for Manufacturing and Engineering*, pages 19–25, 1989.
- [22] Crina Grosan and Ajith Abraham. Rule-based expert systems. 2005.
- [23] V. Akula. *Rule-based systems for medical diagnosis*. IGI Global, 2017.
- [24] Frank Van Harmelen, Vladimir Lifschitz, B Porter, and Peter Struss. Handbook of knowledge representation edited model-based problem solving.
- [25] Bernhard Walzl, Georg Bonczek, and Florian Matthes. Rule-based information extraction: Advantages, limitations, and perspectives. 2018.
- [26] H. Akkermans. Ontologies and their role in knowledge management and e-business modelling. In *International Conference on Enterprise Integration and Modeling Technology*, pages 71–82. Springer, 2002.
- [27] M. Hepp. Possible ontologies: How reality constrains the development of relevant ontologies. *IEEE Internet Computing*, pages 90–96, 2007.
- [28] Liwei Wang, Ruan Jian Xue, Hong-Yi Zhang, Liwei Wang, and Yu-Xi Chen. Research progress of probabilistic graphical models: A survey. 2014.

- [29] Liu Jian-we. Representation theory of probabilistic graphical models. *Computer Science*, 2014.
- [30] Lise Getoor and Ben Taskar. Graphical models in a nutshell. 2007.
- [31] R. Bergmann and K. D. Althoff. Case-based reasoning. *Künstliche Intelligenz*, pages 5–11, 2009.
- [32] C. Jian et al. A review and analysis of case-based reasoning research. In *2015 International Conference on Intelligent Transportation, Big Data and Smart City*, pages 51–55. IEEE, 2015.
- [33] A. Aamodt. Case-based reasoning-an introduction. In *A Newforce in Advanced Systems Development, Unicom Seminars*, pages 9–23, 1995.
- [34] F. Boulmaiz and F. Fessant. Reducing reliance on domain knowledge in case-based reasoning. In *International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems*, pages 3–13. Springer, 2023.
- [35] Thomas R. Hinrichs. Some limitations of feature-based recognition in case-based design. In *International Conference on Case-Based Reasoning*, 1995.
- [36] M. R. Ginsberg. Logic programming. *Communications of the ACM*, pages 933–941, 1985.
- [37] A. Colmerauer and P. Roussel. The birth of prolog. In *HOPL-II*, 1993.
- [38] John W. Lloyd. Foundations of logic programming. In *Symbolic Computation*, 1984.
- [39] Chitta Baral. Knowledge representation, reasoning and declarative problem solving. 2003.
- [40] Michael Gelfond and Vladimir Lifschitz. The stable model semantics for logic programming. In *ICLP/SLP*, 1988.
- [41] Raymond Reiter. A logic for default reasoning. *Artif. Intell.*, 13:81–132, 1987.
- [42] Esra Erdem and Vladimir Lifschitz. Theory and applications of answer set programming. 2002.
- [43] Gopal Gupta, Elmer Salazar, Kyle Marple, Zhuo Chen, and Farhad Shakerin. A case for query-driven predicate answer set programming. In *ARCADE@CADE*, 2017.
- [44] Nicola Leone, Carlo Allocca, Mario Alviano, Francesco Calimeri, Cristina Civili, Roberta Costabile, Alessio Fiorentino, Davide Fuscà, Stefano Germano, Giovanni Labocchetta, Bernardo Cuteri, M. Manna, Simona Perri, Kristian Reale, F. Ricca, Pierfrancesco Veltri, and Jessica Zangari. Enhancing dl原因 for large-scale reasoning. In *International Conference on Logic Programming and Non-Monotonic Reasoning*, 2019.

- [45] Enrico Pontelli. Experiments in parallel execution of answer set programs. In *IEEE International Parallel and Distributed Processing Symposium*, 2001.
- [46] Trevor Rasmusson and John R. Bourne. Interactive storytelling, gamification, and online education: Storytelling made easy. *International Journal on Innovations in Online Education*, 1(3), 2017.
- [47] Tran Cao Son, Enrico Pontelli, and Ngoc-Hieu Nguyen. Planning for multiagent using asp-prolog. In *Computational Logic in Multi-Agent Systems*, 2009.
- [48] M. Gebser and Torsten Schaub. Modeling and language extensions. *AI Mag.*, 37:33–44, 2016.
- [49] Duangtida Athakravi, Ken Satoh, Mark Law, Krysia Broda, and A. Russo. Automated inference of rules with exception from past legal cases using ASP. In *International Conference on Logic Programming and Non-Monotonic Reasoning*, 2015.
- [50] Martin Gebser, Torsten Schaub, Sven Thiele, and Philippe Veber. Detecting inconsistencies in large biological networks with answer set programming. *Theory and Practice of Logic Programming*, 11(2-3):323–360, 2011.
- [51] Tomas Mikolov, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. In *International Conference on Learning Representations*, 2013.
- [52] Alec Radford and Karthik Narasimhan. Improving language understanding by generative pre-training. 2018.
- [53] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics*, 2019.
- [54] Ian H. Witten, Zane Bray, Malika Mahoui, and William John Teahan. Using language models for generic entity extraction. 1999.
- [55] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *ArXiv*, abs/1907.11692, 2019.
- [56] Milena Trajanoska, Riste Stojanov, and Dimitar Trajanov. Enhancing knowledge graph construction using large language models. *ArXiv*, abs/2305.04676, 2023.
- [57] Danning Xie, Byung Kyu Yoo, Nan Jiang, Mijung Kim, Lin Tan, X. Zhang, and Judy S. Lee. Impact of large language models on generating software specifications. *ArXiv*, abs/2306.03324, 2023.

- [58] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context llms struggle with long in-context learning. *ArXiv*, abs/2404.02060, 2024.
- [59] Jane H. Hill. Finding culture in narrative. 2005.
- [60] Li Zhou, Taelin Karidi, Nicolas Garneau, Yong Cao, Wanlong Liu, Wenyu Chen, and Daniel Hershcovich. Does mapo tofu contain coffee? probing llms for food-related cultural knowledge. *ArXiv*, abs/2404.06833, 2024.
- [61] Sonish Sivarajkumar, Mark Kelley, Alyssa Samolyk-Mazzanti, Shyam Visweswaran, and Yanshan Wang. An empirical evaluation of prompting strategies for large language models in zero-shot clinical natural language processing. *ArXiv*, abs/2309.08008, 2023.
- [62] Andrew Drozdov, Nathanael Scharli, Ekin Akyuurek, Nathan Scales, Xinying Song, Xinyun Chen, Olivier Bousquet, and Denny Zhou. Compositional semantic parsing with large language models. *ArXiv*, abs/2209.15003, 2022.
- [63] Zhun Yang, Adam Ishay, and Joohyung Lee. Coupling large language models with logic programming for robust and general reasoning from text. *ArXiv*, abs/2307.07696, 2023.
- [64] Adam Ishay, Zhun Yang, and Joohyung Lee. Leveraging large language models to generate answer set programs. *ArXiv*, abs/2307.07699, 2023.
- [65] Yankai Zeng, Abhiramon Rajasekharan, Parth Padalkar, Kinjal Basu, Joaquín Arias, and Gopal Gupta. Automated interactive domain-specific conversational agents that understand human dialogs. *ArXiv*, abs/2303.08941, 2023.
- [66] Daniela Inclezan. RestKB: A library of commonsense knowledge about dining at a restaurant. In *Proceedings 35th International Conference on Logic Programming (Technical Communications), ICLP 2019 Technical Communications, Las Cruces, NM, USA, September 20-25, 2019*, volume 306 of *EPTCS*, pages 126–139, 2019.
- [67] Nasrin Mostafazadeh, Nathanael Chambers, Xiaodong He, Devi Parikh, Dhruv Batra, Lucy Vanderwende, Pushmeet Kohli, and James Allen. A corpus and cloze evaluation for deeper understanding of commonsense stories. In Kevin Knight, Ani Nenkova, and Owen Rambow, editors, *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 839–849, San Diego, California, June 2016. Association for Computational Linguistics.
- [68] Farraj Albalawi, Sanjeev B. Khanagar, Kiran Iyer, Nora Alhazmi, Afnan Alayyash, Anwar S. Alhazmi, Mohammed Awawdeh, and Oinam Gokulchandra Singh. Evaluating the performance of artificial intelligence-based large language models in orthodontics—a systematic review and meta-analysis. *Applied Sciences*, 15(2):893, January 2025.
- [69] LearnLM Team, Abhinit Modi, Aditya Srikanth Veerubhotla, Aliya Rysbek, Andrea Huber, Ankit Anand, Avishkar Bhoopchand, Brett Wiltshire, Daniel Gillick, Daniel

Kasenberg, Eleni Sgouritsa, Gal Elidan, Hengrui Liu, Holger Winnemoeller, Irina Jurénka, James Cohan, Jennifer She, Julia Wilkowski, Kaiz Alarakya, Kevin R. McKee, Komal Singh, Lisa Wang, Markus Kunesch, Miruna Pîslar, Niv Efron, Parsa Mahmoudieh, Pierre-Alexandre Kamienny, Sara Wiltberger, Shakir Mohamed, Shashank Agarwal, Shubham Milind Phal, Sun Jae Lee, Theofilos Strinopoulos, Wei-Jen Ko, Yael Gold-Zamir, Yael Haramaty, and Yannis Assael. Evaluating gemini in an arena for learning, 2025.

[70] DeepSeek. Deepseek-v3.1 release. DeepSeek API Docs, August 2025.

[71] OpenAI. Gpt-5.1 now available: A smarter, more conversational chatgpt. <https://openai.com/index/gpt-5-1/>, Nov 2025.